

PyTorch 简介

PyTorch 是一个开源的 Python 机器学习库，基于 Torch 库，底层由 C++ 实现，应用于人工智能领域，如计算机视觉和自然语言处理。

PyTorch 最初由 Meta Platforms 的人工智能研究团队开发，现在属于 Linux 基金会的一部分。

许多深度学习软件都是基于 PyTorch 构建的，包括特斯拉自动驾驶、Uber 的 Pyro、Hugging Face 的 Transformers、PyTorch Lightning 和 Catalyst。

PyTorch 主要有两大特征：

- 类似于 NumPy 的张量计算，能在 GPU 或 MPS 等硬件加速器上加速。
- 基于带自动微分系统的深度神经网络。

PyTorch 包括 torch.autograd、torch.nn、torch.optim 等子模块。

PyTorch 包含多种损失函数，包括 MSE（均方误差 = L2 范数）、交叉熵损失和负熵似然损失（对分类器有用）等。



PyTorch 特性

- **动态计算图 (Dynamic Computation Graphs)**：PyTorch 的计算图是动态的，这意味着它们在运行时构建，并且可以随时改变。这为实验和调试提供了极大的灵活性，因为开发者可以逐行执行代码，查看中间结果。
- **自动微分 (Automatic Differentiation)**：PyTorch 的自动微分系统允许开发者轻松地计算梯度，这对于训练深度学习模型至关重要。它通过反向传播算法自动计算出损失函数对模型参数的梯度。
- **张量计算 (Tensor Computation)**：PyTorch 提供了类似于 NumPy 的张量操作，这些操作可以在 CPU 和 GPU 上执行，从而加速计算过程。张量是 PyTorch 中的基本数据结构，用于存储和操作数据。
- **丰富的 API**：PyTorch 提供了大量的预定义层、损失函数和优化算法，这些都是构建深度学习模型的常用组件。
- **多语言支持**：PyTorch 虽然以 Python 为主要接口，但也提供了 C++ 接口，允许更底层的集成和控制。

动态计算图 (Dynamic Computation Graph)

PyTorch 最显著的特点之一是其动态计算图的机制。

与 TensorFlow 的静态计算图 (graph) 不同, PyTorch 在执行时构建计算图, 这意味着在每次计算时, 图都会根据输入数据的形状自动变化。

动态计算图的优点:

- 更加灵活, 特别适用于需要条件判断或递归的场景。
- 方便调试和修改, 能够直接查看中间结果。
- 更接近 Python 编程的风格, 易于上手。

张量 (Tensor) 与自动求导 (Autograd)

PyTorch 中的核心数据结构是 **张量 (Tensor)**, 它是一个多维矩阵, 可以在 CPU 或 GPU 上高效地进行计算。张量的操作支持自动求导 (Autograd) 机制, 使得在反向传播过程中自动计算梯度, 这对于深度学习中的梯度下降优化算法至关重要。

张量 (Tensor) :

- 支持在 CPU 和 GPU 之间进行切换。
- 提供了类似 NumPy 的接口, 支持元素级运算。
- 支持自动求导, 可以方便地进行梯度计算。

自动求导 (Autograd) :

- PyTorch 内置的自动求导引擎, 能够自动追踪所有张量的操作, 并在反向传播时计算梯度。
- 通过 `requires_grad` 属性, 可以指定张量需要计算梯度。
- 支持高效的反向传播, 适用于神经网络的训练。

模型定义与训练

PyTorch 提供了 `torch.nn` 模块, 允许用户通过继承 `nn.Module` 类来定义神经网络模型。使用 `forward` 函数指定前向传播, 自动反向传播 (通过 `autograd`) 和梯度计算也由 PyTorch 内部处理。

神经网络模块 (torch.nn) :

- 提供了常用的层 (如线性层、卷积层、池化层等)。
- 支持定义复杂的神经网络架构 (包括多输入、多输出的网络)。
- 兼容与优化器 (如 `torch.optim`) 一起使用。

GPU 加速

PyTorch 完全支持在 GPU 上运行, 以加速深度学习模型的训练。通过简单的 `.to(device)` 方法, 用户可以将模型和张量转移到 GPU 上进行计算。PyTorch 支持多 GPU 训练, 能够利用 NVIDIA CUDA 技术显著提高计算效率。

GPU 支持:

- 自动选择 GPU 或 CPU。
- 支持通过 CUDA 加速运算。
- 支持多 GPU 并行计算 (`DataParallel` 或 `torch.distributed`)。

生态系统与社区支持

PyTorch 作为一个开源项目，拥有一个庞大的社区和生态系统。它不仅在学术界得到了广泛的应用，也在工业界，特别是在计算机视觉、自然语言处理等领域中得到了广泛部署。PyTorch 还提供了许多与深度学习相关的工具和库，如：

- **torchvision**：用于计算机视觉任务的数据集和模型。
- **torchtext**：用于自然语言处理任务的数据集和预处理工具。
- **torchaudio**：用于音频处理的工具包。
- **PyTorch Lightning**：一种简化 PyTorch 代码的高层库，专注于研究和实验的快速迭代。

与其他框架的对比

PyTorch 由于其灵活性、易用性和社区支持，已经成为很多深度学习研究者和开发者的首选框架。

TensorFlow vs PyTorch

- PyTorch 的动态计算图使得它更加灵活，适合快速实验和研究；而 TensorFlow 的静态计算图在生产环境中更具优化空间。
- PyTorch 在调试时更加方便，TensorFlow 则在部署上更加成熟，支持更广泛的硬件和平台。
- 近年来，TensorFlow 也引入了动态图（如 TensorFlow 2.x），使得两者在功能上趋于接近。
- 其他深度学习框架，如 Keras、Caffe 等也有一定应用，但 PyTorch 由于其灵活性、易用性和社区支持，已经成为很多深度学习研究者和开发者的首选框架。

特性	TensorFlow	PyTorch
开发公司	Google	Facebook (FAIR)
计算图类型	静态计算图（定义后再执行）	动态计算图（定义即执行）
灵活性	低（计算图在编译时构建，不易修改）	高（计算图在执行时动态创建，易于修改和调试）
调试	较难（需要使用 <code>tf.debugging</code> 或外部工具调试）	容易（可以直接在 Python 中进行调试）
易用性	低（较复杂，API 较多，学习曲线较陡峭）	高（API 简洁，语法更加接近 Python，容易上手）
部署	强（支持广泛的硬件，如 TensorFlow Lite、TensorFlow.js）	较弱（部署工具和平台相对较少，虽然有 TensorFlow 支持）
社区支持	很强（成熟且庞大的社区，广泛的教程和文档）	很强（社区活跃，特别是在学术界，快速发展的生态）
模型训练	支持分布式训练，支持多种设备（如 CPU、GPU、TPU）	支持分布式训练，支持多 GPU、CPU 和 TPU
API 层级	高级API: Keras; 低级API: TensorFlow Core	高级API: TorchVision、TorchText 等; 低级API: Torch
性能	高（优化方面成熟，适合生产环境）	高（适合研究和原型开发，生产性能也在提升）
自动求导	通过 <code>tf.GradientTape</code> 实现动态求导（较复杂）	通过 <code>autograd</code> 动态求导（更简洁和直观）
调优与可扩展性	强（支持在多平台上运行，如 TensorFlow Serving 等）	较弱（虽然在学术和实验环境中表现优越，但生产环境支持相对较少）
框架灵活性	较低（TensorFlow 2.x 引入了动态图特性，但仍不完全灵活）	高（动态图带来更高的灵活性）
支持多种语言	支持多种语言（Python, C++, Java, JavaScript, etc.）	主要支持 Python（但也有 C++ API）
兼容性与迁移	TensorFlow 2.x 与旧版本兼容性较好	与 TensorFlow 兼容性差，迁移较难

PyTorch vs NumPy

特性	PyTorch	NumPy
目标	深度学习专用	通用科学计算
GPU 支持	原生支持 CUDA	不直接支持
自动微分	内置自动求导	需要手动计算梯度
神经网络	丰富的神经网络模块	需要从零实现
学习成本	相对较高	相对较低

PyTorch 的历史与发展

PyTorch 的前身是 Torch，这是一个基于 Lua 语言的科学计算框架。随着 Python 在机器学习领域的兴起，Facebook 团队决定将 Torch 的核心思想移植到 Python 上，从而诞生了 PyTorch。

- **2016年**：Facebook 发布 PyTorch 0.1 版本
- **2017年**：PyTorch 0.2 引入分布式训练支持
- **2018年**：PyTorch 1.0 发布，增加了生产部署能力
- **2019年**：PyTorch 1.3 引入移动端支持
- **2020年**：PyTorch 1.6 增加了自动混合精度训练
- **2021年**：PyTorch 1.9 引入 TorchScript 和 C++ 前端
- **2022年**：PyTorch 1.12 优化了性能和稳定性
- **2023年**：PyTorch 2.0 发布，引入编译模式大幅提升性能

PyTorch 安装

PyTorch 是一个流行的深度学习框架，支持 CPU 和 GPU 计算。

支持的操作系统

- **Windows**：Windows 10 或更高版本（64位）
- **macOS**：macOS 10.15 (Catalina) 或更高版本
- **Linux**：主流发行版（Ubuntu 18.04+、CentOS 7+、RHEL 7+等）

Python 版本要求

- **推荐版本**：Python 3.8 - 3.11
- **最低要求**：Python 3.7
- **注意**：Python 3.12+ 支持可能有限，建议使用稳定版本

硬件要求

- **CPU**：支持 SSE4.2 指令集的 x86_64 处理器
- **内存**：至少 4GB RAM（推荐 8GB+）
- **存储**：至少 3GB 可用空间
- **GPU**（可选）：NVIDIA GPU with CUDA Compute Capability 3.5+

CUDA 兼容性（GPU 版本）

PyTorch 版本	支持的 CUDA 版本	推荐 CUDA 版本
2.1.x	11.8, 12.1	12.1
2.0.x	11.7, 11.8	11.8
1.13.x	11.6, 11.7	11.7

安装前的准备工作

检查系统信息

Windows:

```
1 # 检查 windows 版本
2 winver
3
4 # 检查 Python 版本
5 python --version
6
7 # 检查是否有 NVIDIA GPU
8 nvidia-smi
```

macOS

```
1 # 检查 macOS 版本
2 sw_vers
3
4 # 检查 Python 版本
5 python3 --version
6
7 # 检查是否有兼容的 GPU (Apple silicon)
8 system_profiler SPDisplaysDataType
```

Linux

```
1 # 检查发行版信息
2 cat /etc/os-release
3
4 # 检查 Python 版本
5 python3 --version
6
7 # 检查 NVIDIA GPU
8 nvidia-smi
9
10 # 检查 CUDA 版本（如果已安装）
11 nvcc --version
```

Python 环境管理

使用 Anaconda/Miniconda:

```
1 # 下载并安装 Miniconda
2 # windows: https://repo.anaconda.com/miniconda/Miniconda3-latest-windows-
  x86_64.exe
3 # macOS: https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-
  x86_64.sh
4 # Linux: https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-
  x86_64.sh
5
6 # 创建专用环境
7 conda create -n pytorch_env python=3.10
8 conda activate pytorch_env
```

使用 venv (Python 自带)

```
1 # 创建虚拟环境
2 python -m venv pytorch_env
3
4 # 激活环境
5 # windows
6 pytorch_envScriptsactivate
7 # macOS/Linux
8 source pytorch_env/bin/activate
```

安装 PyTorch

PyTorch 官方提供了几种安装方法，可以通过 pip 或 conda 进行安装。

CPU 版本安装

使用 pip 安装 pytorch:

```
1 # 最新稳定版本
2 pip install torch torchvision torchaudio
3
4 # 指定版本
5 pip install torch==2.1.0 torchvision==0.16.0 torchaudio==2.1.0
6
7 # 仅 CPU 版本（更小的安装包）
8 pip install torch torchvision torchaudio --index-url
  https://download.pytorch.org/whl/cpu
```

使用 conda 安装:

如果你使用 Anaconda 或 Miniconda 管理 Python 环境，使用 conda 安装 PyTorch 可能会更加简单和高效。

```
1 # 从 conda-forge 安装
2 conda install pytorch torchvision torchaudio cpuonly -c pytorch
3
4 # 或从 conda-forge 渠道
5 conda install pytorch torchvision torchaudio -c conda-forge
```

如果不了解Anaconda，可以参考：[Anaconda 教程](#)

通过 PyTorch 官网安装

访问 PyTorch 的官方网站 <https://pytorch.org/get-started/locally/>，网站提供了一个方便的工具，可以根据你的系统配置（操作系统、包管理器、Python版本以及CUDA版本）推荐安装命令。

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.5.1)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Conda	Pip		LibTorch	Source
Language	Python			C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	CUDA 12.4	ROCm 6.2	Default
Run this Command:	pip3 install torch torchvision torchaudio				

从源代码安装

如果你需要从源代码安装PyTorch，或者想要尝试最新的开发版本，你可以使用以下命令：

```
1 git clone --recursive https://github.com/pytorch/pytorch
2 cd pytorch
3 python setup.py install
```

这将从 GitHub 克隆 PyTorch 的源代码，并使用 setup.py 进行安装。

GPU 版本安装 (CUDA)

安装 CUDA (如果需要)：


```
1 # Ubuntu/Debian
2 wget
  https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2004/x86_64/c
  uda-ubuntu2004.pin
3 sudo mv cuda-ubuntu2004.pin /etc/apt/preferences.d/cuda-repository-pin-600
4 wget
  https://developer.download.nvidia.com/compute/cuda/12.1.0/local_installers/c
  uda-repo-ubuntu2004-12-1-local_12.1.0-530.30.02-1_amd64.deb
5 sudo dpkg -i cuda-repo-ubuntu2004-12-1-local_12.1.0-530.30.02-1_amd64.deb
6 sudo cp /var/cuda-repo-ubuntu2004-12-1-local/cuda-*-keyring.gpg
  /usr/share/keyrings/
7 sudo apt-get update
8 sudo apt-get -y install cuda
9
10 # CentOS/RHEL
11 sudo yum install -y
  https://developer.download.nvidia.com/compute/cuda/repos/rhel8/x86_64/cuda-
  repo-rhel8-12.1.0-1.x86_64.rpm
12 sudo yum clean all
13 sudo yum -y install cuda
```

安装 PyTorch GPU 版本:

```
1 # CUDA 12.1 版本
2 pip install torch torchvision torchaudio
3
4 # CUDA 11.8 版本
5 pip install torch torchvision torchaudio --index-url
  https://download.pytorch.org/whl/cu118
6
7 # 使用 conda
8 conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c
  nvidia
```

macOS 特殊说明

Apple Silicon (M1/M2) Mac:

```
1 # 使用 Metal Performance Shaders (MPS) 后端
2 pip install torch torchvision torchaudio
3
4 # 验证 MPS 可用性
5 python -c "import torch; print(torch.backends.mps.is_available())"
```

Intel Mac:

```
1 # 标准安装
2 pip install torch torchvision torchaudio
```

验证安装

为了确保 PyTorch 已正确安装，我们可以通过执行以下 PyTorch 代码来验证是否安装成功：

```
1
2 import torch
3
4 # 当前安装的 PyTorch 库的版本
5 print(torch.__version__)
6 # 检查 CUDA 是否可用，即你的系统有 NVIDIA 的 GPU
7 print(torch.cuda.is_available())
```

如果 `torch.cuda.is_available()` 输出 `True`，则说明 PyTorch 成功识别到你的 GPU。

一个简单的实例，构建一个随机初始化的张量：

```
1
2 import torch
3 x = torch.rand(5, 3)
4 print(x)
```

如果安装成功，输出结果类似如下：

```
1 tensor([[0.3380, 0.3845, 0.3217],
2         [0.8337, 0.9050, 0.2650],
3         [0.2979, 0.7141, 0.9069],
4         [0.1449, 0.1132, 0.1375],
5         [0.4675, 0.3947, 0.1426]])
```

PyTorch 基础

PyTorch 是一个开源的深度学习框架，以其灵活性和动态计算图而广受欢迎。

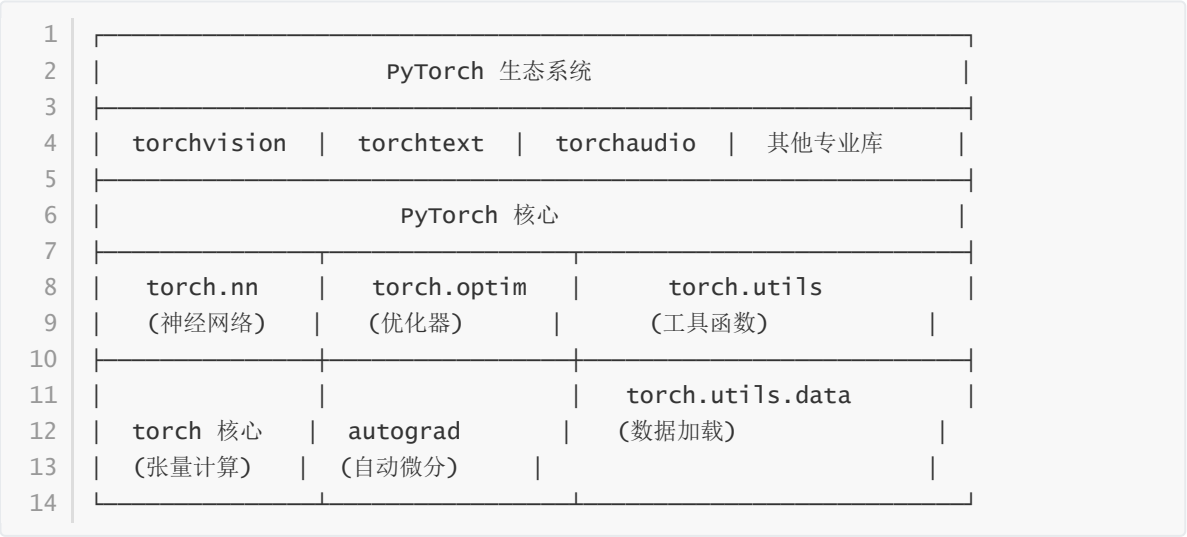
PyTorch 主要有以下几个基础概念：张量（Tensor）、自动求导（Autograd）、神经网络模块（`nn.Module`）、优化器（`optim`）等。

- **张量 (Tensor)**：PyTorch 的核心数据结构，支持多维数组，并可以在 CPU 或 GPU 上进行加速计算。
- **自动求导 (Autograd)**：PyTorch 提供了自动求导功能，可以轻松计算模型的梯度，便于进行反向传播和优化。
- **神经网络 (nn.Module)**：PyTorch 提供了简单且强大的 API 来构建神经网络模型，可以方便地进行前向传播和模型定义。
- **优化器 (Optimizers)**：使用优化器（如 Adam、SGD 等）来更新模型的参数，使得损失最小化。
- **设备 (Device)**：可以将模型和张量移动到 GPU 上以加速计算。

PyTorch 架构总览

PyTorch 采用模块化设计，由多个相互协作的核心组件构成。理解这些组件的作用和相互关系，是掌握 PyTorch 的关键。

PyTorch 架构图



PyTorch 采用**分层架构**设计，从上层到底层依次为：

1、Python API（顶层）

- `torch`：核心张量计算（类似NumPy，支持GPU）。
- `torch.nn`：神经网络层、损失函数等。
- `torch.autograd`：自动微分（反向传播）。
- 开发者直接调用的接口，简单易用。

2、C++核心（中层）

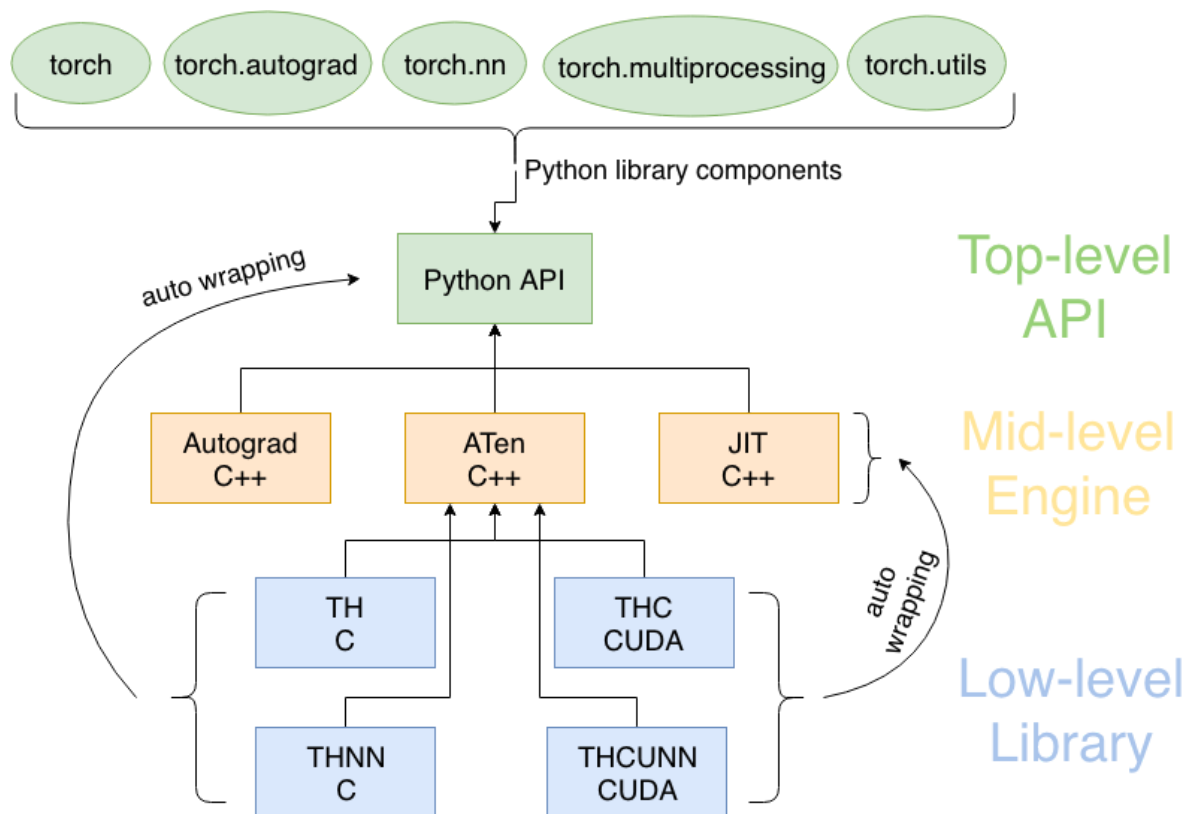
- **ATen**：张量运算核心库（400+操作）。
- **JIT**：即时编译优化模型。
- **Autograd引擎**：自动微分的底层实现。
- 高性能计算，连接Python与底层硬件。

3、基础库（底层）

- **TH/THNN**：C语言实现的基础张量和神经网络操作。
- **THC/THCUNN**：对应的CUDA（GPU）版本。
- 直接操作硬件（CPU/GPU），极致优化速度。

执行流程：

Python代码 → C++核心计算 → 底层CUDA/C库加速 → 返回结果。
既保持易用性，又确保高性能。



张量 (Tensor)

张量 (Tensor) 是 PyTorch 中的核心数据结构，用于存储和操作多维数组。

张量可以视为一个多维数组，支持加速计算的操作。

在 PyTorch 中，张量的概念类似于 NumPy 中的数组，但是 PyTorch 的张量可以运行在不同的设备上，比如 CPU 和 GPU，这使得它们非常适合于进行大规模并行计算，特别是在深度学习领域。

- **维度 (Dimensionality)：** 张量的维度指的是数据的多维数组结构。例如，一个标量 (0维张量) 是一个单独的数字，一个向量 (1维张量) 是一个一维数组，一个矩阵 (2维张量) 是一个二维数组，以此类推。
- **形状 (Shape)：** 张量的形状是指每个维度上的大小。例如，一个形状为 (3, 4) 的张量意味着它有3行4列。
- **数据类型 (Dtype)：** 张量中的数据类型定义了存储每个元素所需的内存大小和解释方式。PyTorch支持多种数据类型，包括整数型 (如 `torch.int8`、`torch.int32`)、浮点型 (如 `torch.float32`、`torch.float64`) 和布尔型 (`torch.bool`)。

张量创建：

```
1 import torch
2
3
4 # 创建一个 2x3 的全 0 张量
5 a = torch.zeros(2, 3)
6 print(a)
7
8 # 创建一个 2x3 的全 1 张量
9 b = torch.ones(2, 3)
10 print(b)
11
12 # 创建一个 2x3 的随机数张量
13 c = torch.randn(2, 3)
14 print(c)
```

```

15
16 # 从 NumPy 数组创建张量
17 import numpy as np
18 numpy_array = np.array([[1, 2], [3, 4]])
19 tensor_from_numpy = torch.from_numpy(numpy_array)
20 print(tensor_from_numpy)
21
22 # 在指定设备（CPU/GPU）上创建张量
23 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
24 d = torch.randn(2, 3, device=device)
25 print(d)

```

输出结果类似如下：

```

1  tensor([[0., 0., 0.],
2         [0., 0., 0.]])
3  tensor([[1., 1., 1.],
4         [1., 1., 1.]])
5  tensor([[ 1.0189, -0.5718, -1.2814],
6         [-0.5865,  1.0855,  1.1727]])
7  tensor([[1, 2],
8         [3, 4]])
9  tensor([[ -0.3360,  0.2203,  1.3463],
10         [-0.5982, -0.2704,  0.5429]])

```

常用张量操作：

```

1  # 张量相加
2  e = torch.randn(2, 3)
3  f = torch.randn(2, 3)
4  print(e + f)
5
6  # 逐元素乘法
7  print(e * f)
8
9  # 张量的转置
10 g = torch.randn(3, 2)
11 print(g.t()) # 或者 g.transpose(0, 1)
12
13 # 张量的形状
14 print(g.shape) # 返回形状

```

张量与设备

PyTorch 张量可以存在于不同的设备上，包括CPU和GPU，你可以将张量移动到 GPU 上以加速计算：

```

1  if torch.cuda.is_available():
2      tensor_gpu = tensor_from_list.to('cuda') # 将张量移动到GPU

```

梯度和自动微分

PyTorch的张量支持自动微分，这是深度学习中的关键特性。当你创建一个需要梯度的张量时，PyTorch可以自动计算其梯度：

```
1 # 创建一个需要梯度的张量
2 tensor_requires_grad = torch.tensor([1.0], requires_grad=True)
3
4 # 进行一些操作
5 tensor_result = tensor_requires_grad * 2
6
7 # 计算梯度
8 tensor_result.backward()
9 print(tensor_requires_grad.grad) # 输出梯度
```

内存和性能

PyTorch 张量还提供了一些内存管理功能，比如.clone()、.detach() 和 .to() 方法，它们可以帮助你优化内存使用和提高性能。

自动求导（Autograd）

自动求导（Automatic Differentiation，简称Autograd）是深度学习框架中的一个核心特性，它允许计算机自动计算数学函数的导数。

在深度学习中，自动求导主要用于两个方面：**一是在训练神经网络时计算梯度，二是进行反向传播算法的实现。**

自动求导基于链式法则（Chain Rule），这是一个用于计算复杂函数导数的数学法则。链式法则表明，复合函数的导数是其各个组成部分导数的乘积。在深度学习中，模型通常是由许多层组成的复杂函数，自动求导能够高效地计算这些层的梯度。

动态图与静态图：

- **动态图（Dynamic Graph）**：在动态图中，计算图在运行时动态构建。每次执行操作时，计算图都会更新，这使得调试和修改模型变得更加容易。PyTorch使用的是动态图。
- **静态图（Static Graph）**：在静态图中，计算图在开始执行之前构建完成，并且不会改变。TensorFlow最初使用的是静态图，但后来也支持动态图。

PyTorch 提供了自动求导功能，通过 autograd 模块来自动计算梯度。

torch.Tensor 对象有一个 requires_grad 属性，用于指示是否需要计算该张量的梯度。

当你创建一个 requires_grad=True 的张量时，PyTorch 会自动跟踪所有对它的操作，以便在之后计算梯度。

创建需要梯度的张量：

```

1 # 创建一个需要计算梯度的张量
2 x = torch.randn(2, 2, requires_grad=True)
3 print(x)
4
5 # 执行某些操作
6 y = x + 2
7 z = y * y * 3
8 out = z.mean()
9
10 print(out)

```

输出结果类似如下：

```

1 tensor([[0., 0., 0.],
2         [0., 0., 0.]])
3 tensor([[1., 1., 1.],
4         [1., 1., 1.]])
5 tensor([[ 1.0189, -0.5718, -1.2814],
6         [-0.5865,  1.0855,  1.1727]])
7 tensor([[1, 2],
8         [3, 4]])
9 tensor([[ -0.3360,  0.2203,  1.3463],
10         [-0.5982, -0.2704,  0.5429]])
11 tianqixin@Mac-mini runoob-test % python3 test.py
12 tensor([[ -0.1908,  0.2811],
13         [ 0.8068,  0.8002]], requires_grad=True)
14 tensor(18.1469, grad_fn=<MeanBackward0>)

```

反向传播 (Backpropagation)

一旦定义了计算图，可以通过 `.backward()` 方法来计算梯度。

```

1 # 反向传播，计算梯度
2 out.backward()
3
4 # 查看 x 的梯度
5 print(x.grad)

```

在神经网络训练中，自动求导主要用于实现反向传播算法。

反向传播是一种通过计算损失函数关于网络参数的梯度来训练神经网络的方法。在每次迭代中，网络的前向传播会计算输出和损失，然后反向传播会计算损失关于每个参数的梯度，并使用这些梯度来更新参数。

停止梯度计算

如果你不希望某些张量的梯度被计算（例如，当你不需要反向传播时），可以使用 `torch.no_grad()` 或设置 `requires_grad=False`。

```

1 # 使用 torch.no_grad() 禁用梯度计算
2 with torch.no_grad():
3     y = x * 2

```

神经网络 (nn.Module)

神经网络是一种模仿人脑神经元连接的计算模型，由多层节点（神经元）组成，用于学习数据之间的复杂模式和关系。

神经网络通过调整神经元之间的连接权重来优化预测结果，这一过程涉及前向传播、损失计算、反向传播和参数更新。

神经网络的类型包括前馈神经网络、卷积神经网络（CNN）、循环神经网络（RNN）和长短期记忆网络（LSTM），它们在图像识别、语音处理、自然语言处理等多个领域都有广泛应用。

PyTorch 提供了一个非常方便的接口来构建神经网络模型，即 `torch.nn.Module`。

我们可以继承 `nn.Module` 类并定义自己的网络层。

创建一个简单的神经网络：

```
1 import torch.nn as nn
2 import torch.optim as optim
3
4 # 定义一个简单的全连接神经网络
5 class SimpleNN(nn.Module):
6     def __init__(self):
7         super(SimpleNN, self).__init__()
8         self.fc1 = nn.Linear(2, 2) # 输入层到隐藏层
9         self.fc2 = nn.Linear(2, 1) # 隐藏层到输出层
10
11     def forward(self, x):
12         x = torch.relu(self.fc1(x)) # ReLU 激活函数
13         x = self.fc2(x)
14         return x
15
16 # 创建网络实例
17 model = SimpleNN()
18
19 # 打印模型结构
20 print(model)
```

输出结果：

```
1 SimpleNN(
2   (fc1): Linear(in_features=2, out_features=2, bias=True)
3   (fc2): Linear(in_features=2, out_features=1, bias=True)
4 )
```

训练过程：

- 前向传播 (Forward Propagation)**：在前向传播阶段，输入数据通过网络层传递，每层应用权重和激活函数，直到产生输出。
- 计算损失 (Calculate Loss)**：根据网络的输出和真实标签，计算损失函数的值。
- 反向传播 (Backpropagation)**：反向传播利用自动求导技术计算损失函数关于每个参数的梯度。
- 参数更新 (Parameter Update)**：使用优化器根据梯度更新网络的权重和偏置。
- 迭代 (Iteration)**：重复上述过程，直到模型在训练数据上的性能达到满意的水平。

前向传播与损失计算

```
1 # 随机输入
2 x = torch.randn(1, 2)
3
4 # 前向传播
5 output = model(x)
6 print(output)
7
8 # 定义损失函数（例如均方误差 MSE）
9 criterion = nn.MSELoss()
10
11 # 假设目标值为 1
12 target = torch.randn(1, 1)
13
14 # 计算损失
15 loss = criterion(output, target)
16 print(loss)
```

优化器 (Optimizers)

优化器在训练过程中更新神经网络的参数，以减少损失函数的值。

PyTorch 提供了多种优化器，例如 SGD、Adam 等。

使用优化器进行参数更新：

```
1
2 # 定义优化器（使用 Adam 优化器）
3 optimizer = optim.Adam(model.parameters(), lr=0.001)
4
5 # 训练步骤
6 optimizer.zero_grad() # 清空梯度
7 loss.backward() # 反向传播
8 optimizer.step() # 更新参数
```

训练模型

训练模型是机器学习和深度学习中的核心过程，旨在通过大量数据学习模型参数，以便模型能够对新的、未见过的数据做出准确的预测。

训练模型通常包括以下几个步骤：

1. 数据准备：

- 收集和处理数据，包括清洗、标准化和归一化。
- 将数据分为训练集、验证集和测试集。

2. 定义模型：

- 选择模型架构，例如决策树、神经网络等。
- 初始化模型参数（权重和偏置）。

3. 选择损失函数：

- 根据任务类型（如分类、回归）选择合适的损失函数。

4. 选择优化器：

- 选择一个优化算法，如SGD、Adam等，来更新模型参数。

5. 前向传播:

- 在每次迭代中, 将输入数据通过模型传递, 计算预测输出。

6. 计算损失:

- 使用损失函数评估预测输出与真实标签之间的差异。

7. 反向传播:

- 利用自动求导计算损失相对于模型参数的梯度。

8. 参数更新:

- 根据计算出的梯度和优化器的策略更新模型参数。

9. 迭代优化:

- 重复步骤5-8, 直到模型在验证集上的性能不再提升或达到预定的迭代次数。

10. 评估和测试:

- 使用测试集评估模型的最终性能, 确保模型没有过拟合。

11. 模型调优:

- 根据模型在测试集上的表现进行调参, 如改变学习率、增加正则化等。

12. 部署模型:

- 将训练好的模型部署到生产环境中, 用于实际的预测任务。

```
1
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5
6 # 1. 定义一个简单的神经网络模型
7 class SimpleNN(nn.Module):
8     def __init__(self):
9         super(SimpleNN, self).__init__()
10        self.fc1 = nn.Linear(2, 2) # 输入层到隐藏层
11        self.fc2 = nn.Linear(2, 1) # 隐藏层到输出层
12
13        def forward(self, x):
14            x = torch.relu(self.fc1(x)) # ReLU 激活函数
15            x = self.fc2(x)
16            return x
17
18 # 2. 创建模型实例
19 model = SimpleNN()
20
21 # 3. 定义损失函数和优化器
22 criterion = nn.MSELoss() # 均方误差损失函数
23 optimizer = optim.Adam(model.parameters(), lr=0.001) # Adam 优化器
24
25 # 4. 假设我们有训练数据 X 和 Y
26 x = torch.randn(10, 2) # 10 个样本, 2 个特征
27 y = torch.randn(10, 1) # 10 个目标值
28
29 # 5. 训练循环
30 for epoch in range(100): # 训练 100 轮
31     optimizer.zero_grad() # 清空之前的梯度
32     output = model(x) # 前向传播
```

```
33     loss = criterion(output, Y) # 计算损失
34     loss.backward() # 反向传播
35     optimizer.step() # 更新参数
36
37     # 每 10 轮输出一次损失
38     if (epoch+1) % 10 == 0:
39         print(f'Epoch [{epoch+1}/100], Loss: {loss.item():.4f}')
```

输出结果如下：

```
1 Epoch [10/100], Loss: 1.7180
2 Epoch [20/100], Loss: 1.6352
3 Epoch [30/100], Loss: 1.5590
4 Epoch [40/100], Loss: 1.4896
5 Epoch [50/100], Loss: 1.4268
6 Epoch [60/100], Loss: 1.3704
7 Epoch [70/100], Loss: 1.3198
8 Epoch [80/100], Loss: 1.2747
9 Epoch [90/100], Loss: 1.2346
10 Epoch [100/100], Loss: 1.1991
```

在每 10 轮，程序会输出当前的损失值，帮助我们跟踪模型的训练进度。随着训练的进行，损失值应该会逐渐降低，表示模型在不断学习并优化其参数。

训练模型是一个迭代的过程，需要不断地调整和优化，直到达到满意的性能。这个过程涉及到大量的实验和调优，目的是使模型在新的、未见过的数据上也能有良好的泛化能力。

设备 (Device)

PyTorch 允许你将模型和数据移动到 GPU 上进行加速。

使用 `torch.device` 来指定计算设备。

将模型和数据移至 GPU：

```
1
2 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
3
4 # 将模型移动到设备
5 model.to(device)
6
7 # 将数据移动到设备
8 X = X.to(device)
9 Y = Y.to(device)
```

在训练过程中，所有张量和模型都应该移到同一个设备上（要么都在 CPU 上，要么都在 GPU 上）。

PyTorch 张量 (Tensor)

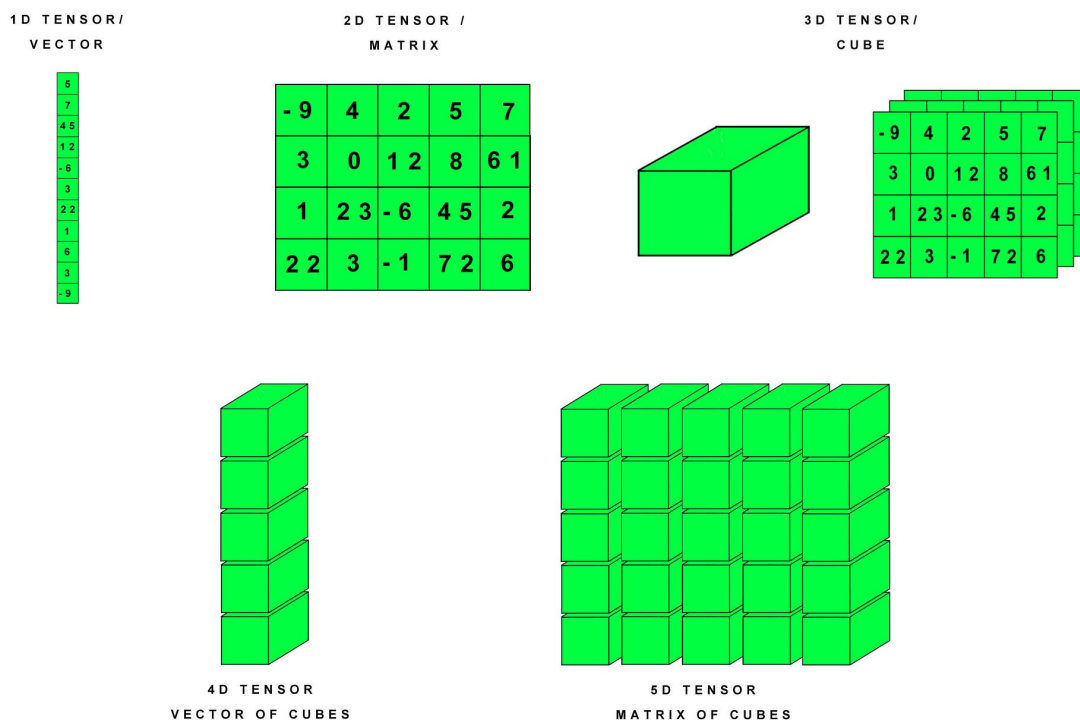
张量是一个多维数组，可以是标量、向量、矩阵或更高维度的数据结构。

在 PyTorch 中，张量 (Tensor) 是数据的核心表示形式，类似于 NumPy 的多维数组，但具有更强大的功能，例如支持 GPU 加速和自动梯度计算。

张量支持多种数据类型（整型、浮点型、布尔型等）。

张量可以存储在 CPU 或 GPU 中，GPU 张量可显著加速计算。

下图展示了不同维度的张量（Tensor）在 PyTorch 中的表示方法：



说明：

- **1D Tensor / Vector（一维张量/向量）**：最基本的张量形式，可以看作是一个数组，图中的例子是一个包含 10 个元素的向量。
- **2D Tensor / Matrix（二维张量/矩阵）**：二维数组，通常用于表示矩阵，图中的例子是一个 4x5 的矩阵，包含了 20 个元素。
- **3D Tensor / Cube（三维张量/立方体）**：三维数组，可以看作是由多个矩阵堆叠而成的立方体，图中的例子展示了一个 3x4x5 的立方体，其中每个 5x5 的矩阵代表立方体的一个“层”。
- **4D Tensor / Vector of Cubes（四维张量/立方体向量）**：四维数组，可以看作是由多个立方体组成的向量，图中的例子没有具体数值，但可以理解为一个包含多个 3D 张量的集合。
- **5D Tensor / Matrix of Cubes（五维张量/立方体矩阵）**：五维数组，可以看作是由多个 4D 张量组成的矩阵，图中的例子同样没有具体数值，但可以理解为一个包含多个 4D 张量的集合。

创建张量

张量创建的方式有：

方法	说明	示例代码
<code>torch.tensor(data)</code>	从 Python 列表或 NumPy 数组创建张量。	<code>x = torch.tensor([[1, 2], [3, 4]])</code>
<code>torch.zeros(size)</code>	创建一个全为零的张量。	<code>x = torch.zeros((2, 3))</code>
<code>torch.ones(size)</code>	创建一个全为 1 的张量。	<code>x = torch.ones((2, 3))</code>
<code>torch.empty(size)</code>	创建一个未初始化的张量。	<code>x = torch.empty((2, 3))</code>
<code>torch.rand(size)</code>	创建一个服从均匀分布的随机张量，值在 [0, 1)。	<code>x = torch.rand((2, 3))</code>
<code>torch.randn(size)</code>	创建一个服从正态分布的随机张量，均值为 0，标准差为 1。	<code>x = torch.randn((2, 3))</code>
<code>torch.arange(start, end, step)</code>	创建一个一维序列张量，类似于 Python 的 <code>range</code> 。	<code>x = torch.arange(0, 10, 2)</code>
<code>torch.linspace(start, end, steps)</code>	创建一个在指定范围内等间隔的序列张量。	<code>x = torch.linspace(0, 1, 5)</code>
<code>torch.eye(size)</code>	创建一个单位矩阵（对角线为 1，其他为 0）。	<code>x = torch.eye(3)</code>
<code>torch.from_numpy(ndarray)</code>	将 NumPy 数组转换为张量。	<code>x = torch.from_numpy(np.array([1, 2, 3]))</code>

使用 `torch.tensor()` 函数，你可以将一个列表或数组转换为张量：

```

1 import torch
2
3 tensor = torch.tensor([1, 2, 3])
4 print(tensor)

```

输出如下：

```

1 tensor([1, 2, 3])

```

如果你有一个 NumPy 数组，可以使用 `torch.from_numpy()` 将其转换为张量：

```
1 import numpy as np
2
3 np_array = np.array([1, 2, 3])
4 tensor = torch.from_numpy(np_array)
5 print(tensor)
```

输出如下：

```
1 tensor([1, 2, 3])
```

创建 2D 张量（矩阵）：

```
1 import torch
2
3 tensor_2d = torch.tensor([
4     [-9, 4, 2, 5, 7],
5     [3, 0, 12, 8, 6],
6     [1, 23, -6, 45, 2],
7     [22, 3, -1, 72, 6]
8 ])
9 print("2D Tensor (Matrix):n", tensor_2d)
10 print("Shape:", tensor_2d.shape) # 形状
```

输出如下：

```
1 2D Tensor (Matrix):
2 tensor([[ -9,  4,  2,  5,  7],
3         [ 3,  0, 12,  8,  6],
4         [ 1, 23, -6, 45,  2],
5         [22,  3, -1, 72,  6]])
6 Shape: torch.Size([4, 5])
```

其他维度的创建：

```
1 # 创建 3D 张量（立方体）
2 tensor_3d = torch.stack([tensor_2d, tensor_2d + 10, tensor_2d - 5]) # 堆叠 3
   个 2D 张量
3 print("3D Tensor (Cube):n", tensor_3d)
4 print("Shape:", tensor_3d.shape) # 形状
5
6 # 创建 4D 张量（向量的立方体）
7 tensor_4d = torch.stack([tensor_3d, tensor_3d + 100]) # 堆叠 2 个 3D 张量
8 print("4D Tensor (Vector of Cubes):n", tensor_4d)
9 print("Shape:", tensor_4d.shape) # 形状
10
11 # 创建 5D 张量（矩阵的立方体）
12 tensor_5d = torch.stack([tensor_4d, tensor_4d + 1000]) # 堆叠 2 个 4D 张量
13 print("5D Tensor (Matrix of Cubes):n", tensor_5d)
14 print("Shape:", tensor_5d.shape) # 形状
```

张量的属性

张量的属性如下表：

属性	说明	示例
<code>.shape</code>	获取张量的形状	<code>tensor.shape</code>
<code>.size()</code>	获取张量的形状	<code>tensor.size()</code>
<code>.dtype</code>	获取张量的数据类型	<code>tensor.dtype</code>
<code>.device</code>	查看张量所在的设备 (CPU/GPU)	<code>tensor.device</code>
<code>.dim()</code>	获取张量的维度数	<code>tensor.dim()</code>
<code>.requires_grad</code>	是否启用梯度计算	<code>tensor.requires_grad</code>
<code>.numel()</code>	获取张量中的元素总数	<code>tensor.numel()</code>
<code>.is_cuda</code>	检查张量是否在 GPU 上	<code>tensor.is_cuda</code>
<code>.T</code>	获取张量的转置 (适用于 2D 张量)	<code>tensor.T</code>
<code>.item()</code>	获取单元素张量的值	<code>tensor.item()</code>
<code>.is_contiguous()</code>	检查张量是否连续存储	<code>tensor.is_contiguous()</code>

```
1 import torch
2
3 # 创建一个 2D 张量
4 tensor = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
5
6 # 张量的属性
7 print("Tensor:\n", tensor)
8 print("Shape:", tensor.shape) # 获取形状
9 print("Size:", tensor.size()) # 获取形状 (另一种方法)
10 print("Data Type:", tensor.dtype) # 数据类型
11 print("Device:", tensor.device) # 设备
12 print("Dimensions:", tensor.dim()) # 维度数
13 print("Total Elements:", tensor.numel()) # 元素总数
14 print("Requires Grad:", tensor.requires_grad) # 是否启用梯度
15 print("Is CUDA:", tensor.is_cuda) # 是否在 GPU 上
16 print("Is Contiguous:", tensor.is_contiguous()) # 是否连续存储
17
18 # 获取单元素值
19 single_value = torch.tensor(42)
20 print("Single Element Value:", single_value.item())
21
22 # 转置张量
23 tensor_T = tensor.T
24 print("Transposed Tensor:\n", tensor_T)
```

输出结果:

```
1 Tensor:
2   tensor([[1., 2., 3.],
3           [4., 5., 6.]])
4 Shape: torch.Size([2, 3])
5 Size: torch.Size([2, 3])
6 Data Type: torch.float32
```

```
7 | Device: cpu
8 | Dimensions: 2
9 | Total Elements: 6
10 | Requires Grad: False
11 | Is CUDA: False
12 | Is Contiguous: True
13 | Single Element Value: 42
14 | Transposed Tensor:
15 |   tensor([[1., 4.],
16 |           [2., 5.],
17 |           [3., 6.]])
```

张量的操作

张量操作方法说明如下。

基础操作：

操作	说明	示例代码
<code>+, -, *, /</code>	元素级加法、减法、乘法、除法。	<code>z = x + y</code>
<code>torch.matmul(x, y)</code>	矩阵乘法。	<code>z = torch.matmul(x, y)</code>
<code>torch.dot(x, y)</code>	向量点积（仅适用于 1D 张量）。	<code>z = torch.dot(x, y)</code>
<code>torch.sum(x)</code>	求和。	<code>z = torch.sum(x)</code>
<code>torch.mean(x)</code>	求均值。	<code>z = torch.mean(x)</code>
<code>torch.max(x)</code>	求最大值。	<code>z = torch.max(x)</code>
<code>torch.min(x)</code>	求最小值。	<code>z = torch.min(x)</code>
<code>torch.argmax(x, dim)</code>	返回最大值的索引（指定维度）。	<code>z = torch.argmax(x, dim=1)</code>
<code>torch.softmax(x, dim)</code>	计算 softmax（指定维度）。	<code>z = torch.softmax(x, dim=1)</code>

形状操作

操作	说明	示例代码
<code>x.view(shape)</code>	改变张量的形状（不改变数据）。	<code>z = x.view(3, 4)</code>
<code>x.reshape(shape)</code>	类似于 <code>view</code> ，但更灵活。	<code>z = x.reshape(3, 4)</code>
<code>x.t()</code>	转置矩阵。	<code>z = x.t()</code>
<code>x.unsqueeze(dim)</code>	在指定维度添加一个维度。	<code>z = x.unsqueeze(0)</code>
<code>x.squeeze(dim)</code>	去掉指定维度为 1 的维度。	<code>z = x.squeeze(0)</code>
<code>torch.cat((x, y), dim)</code>	按指定维度连接多个张量。	<code>z = torch.cat((x, y), dim=1)</code>

```

1  import torch
2
3  # 创建一个 2D 张量
4  tensor = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
5  print("原始张量:n", tensor)
6
7  # 1. **索引和切片操作**
8  print("n【索引和切片】")
9  print("获取第一行:", tensor[0]) # 获取第一行
10 print("获取第一行第一列的元素:", tensor[0, 0]) # 获取特定元素
11 print("获取第二列的所有元素:", tensor[:, 1]) # 获取第二列所有元素
12
13 # 2. **形状变换操作**
14 print("n【形状变换】")
15 reshaped = tensor.view(3, 2) # 改变张量形状为 3x2
16 print("改变形状后的张量:n", reshaped)
17 flattened = tensor.flatten() # 将张量展平成一维
18 print("展平后的张量:n", flattened)
19
20 # 3. **数学运算操作**
21 print("n【数学运算】")
22 tensor_add = tensor + 10 # 张量加法
23 print("张量加 10:n", tensor_add)
24 tensor_mul = tensor * 2 # 张量乘法
25 print("张量乘 2:n", tensor_mul)
26 tensor_sum = tensor.sum() # 计算所有元素的和
27 print("张量元素的和:", tensor_sum.item())
28
29 # 4. **与其他张量的操作**
30 print("n【与其他张量操作】")
31 tensor2 = torch.tensor([[1, 1, 1], [1, 1, 1]], dtype=torch.float32)
32 print("另一个张量:n", tensor2)
33 tensor_dot = torch.matmul(tensor, tensor2.T) # 张量矩阵乘法
34 print("矩阵乘法结果:n", tensor_dot)
35
36 # 5. **条件判断和筛选**
37 print("n【条件判断和筛选】")
38 mask = tensor > 3 # 创建一个布尔掩码
39 print("大于 3 的元素的布尔掩码:n", mask)
40 filtered_tensor = tensor[tensor > 3] # 筛选出符合条件的元素
41 print("大于 3 的元素:n", filtered_tensor)

```

输出结果：

```
1  原始张量：
2      tensor([[1., 2., 3.],
3              [4., 5., 6.]])
4
5  【索引和切片】
6  获取第一行：tensor([1., 2., 3.])
7  获取第一行第一列的元素：tensor(1.)
8  获取第二列的所有元素：tensor([2., 5.])
9
10 【形状变换】
11 改变形状后的张量：
12      tensor([[1., 2.],
13              [3., 4.],
14              [5., 6.]])
15 展平后的张量：
16      tensor([1., 2., 3., 4., 5., 6.])
17
18 【数学运算】
19 张量加 10：
20      tensor([[11., 12., 13.],
21              [14., 15., 16.]])
22 张量乘 2：
23      tensor([[ 2.,  4.,  6.],
24              [ 8., 10., 12.]])
25 张量元素的和：21.0
26
27 【与其他张量操作】
28 另一个张量：
29      tensor([[1., 1., 1.],
30              [1., 1., 1.]])
31 矩阵乘法结果：
32      tensor([[ 6.,  6.],
33              [15., 15.]])
34
35 【条件判断和筛选】
36 大于 3 的元素的布尔掩码：
37      tensor([[False, False, False],
38              [ True,  True,  True]])
39 大于 3 的元素：
40      tensor([4., 5., 6.]])
```

张量的 GPU 加速

将张量转移到 GPU：

```
1  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2  x = torch.tensor([1.0, 2.0, 3.0], device=device)
```

检查 GPU 是否可用：

```
1  torch.cuda.is_available() # 返回 True 或 False
```

张量与 NumPy 的互操作

张量与 NumPy 的互操作如下表所示：

操作	说明	示例代码
<code>torch.from_numpy(ndarray)</code>	将 NumPy 数组转换为张量。	<code>x = torch.from_numpy(np_array)</code>
<code>x.numpy()</code>	将张量转换为 NumPy 数组（仅限 CPU 张量）。	<code>np_array = x.numpy()</code>

```
1
2 import torch
3 import numpy as np
4
5 # 1. NumPy 数组转换为 PyTorch 张量
6 print("1. NumPy 转为 PyTorch 张量")
7 numpy_array = np.array([[1, 2, 3], [4, 5, 6]])
8 print("NumPy 数组:n", numpy_array)
9
10 # 使用 torch.from_numpy() 将 NumPy 数组转换为张量
11 tensor_from_numpy = torch.from_numpy(numpy_array)
12 print("转换后的 PyTorch 张量:n", tensor_from_numpy)
13
14 # 修改 NumPy 数组，观察张量的变化（共享内存）
15 numpy_array[0, 0] = 100
16 print("修改后的 NumPy 数组:n", numpy_array)
17 print("PyTorch 张量也会同步变化:n", tensor_from_numpy)
18
19 # 2. PyTorch 张量转换为 NumPy 数组
20 print("n2. PyTorch 张量转为 NumPy 数组")
21 tensor = torch.tensor([[7, 8, 9], [10, 11, 12]], dtype=torch.float32)
22 print("PyTorch 张量:n", tensor)
23
24 # 使用 tensor.numpy() 将张量转换为 NumPy 数组
25 numpy_from_tensor = tensor.numpy()
26 print("转换后的 NumPy 数组:n", numpy_from_tensor)
27
28 # 修改张量，观察 NumPy 数组的变化（共享内存）
29 tensor[0, 0] = 77
30 print("修改后的 PyTorch 张量:n", tensor)
31 print("NumPy 数组也会同步变化:n", numpy_from_tensor)
32
33 # 3. 注意：不共享内存的情况（需要复制数据）
34 print("n3. 使用 clone() 保证独立数据")
35 tensor_independent = torch.tensor([[13, 14, 15], [16, 17, 18]],
36                                     dtype=torch.float32)
37 numpy_independent = tensor_independent.clone().numpy() # 使用 clone 复制数据
38
39 print("原始张量:n", tensor_independent)
40 tensor_independent[0, 0] = 0 # 修改张量数据
41 print("修改后的张量:n", tensor_independent)
42 print("NumPy 数组（不会同步变化）:n", numpy_independent)
```

输出结果：

```
1  1. NumPy 转为 PyTorch 张量
2  NumPy 数组：
3      [[1 2 3]
4      [4 5 6]]
5  转换后的 PyTorch 张量：
6      tensor([[1, 2, 3],
7              [4, 5, 6]])
8
9  修改后的 NumPy 数组：
10     [[100  2  3]
11     [ 4  5  6]]
12  PyTorch 张量也会同步变化：
13     tensor([[100,  2,  3],
14             [ 4,  5,  6]])
15
16  2. PyTorch 张量转为 NumPy 数组
17  PyTorch 张量：
18     tensor([[ 7.,  8.,  9.],
19             [10., 11., 12.]])
20  转换后的 NumPy 数组：
21     [[ 7.  8.  9.]
22     [10. 11. 12.]]
23
24  修改后的 PyTorch 张量：
25     tensor([[77.,  8.,  9.],
26             [10., 11., 12.]])
27  NumPy 数组也会同步变化：
28     [[77.  8.  9.]
29     [10. 11. 12.]]
30
31  3. 使用 clone() 保证独立数据
32  原始张量：
33     tensor([[13., 14., 15.],
34             [16., 17., 18.]])
35  修改后的张量：
36     tensor([[ 0., 14., 15.],
37             [16., 17., 18.]])
38  NumPy 数组（不会同步变化）：
39     [[13. 14. 15.]
40     [16. 17. 18.]]
```

PyTorch 神经网络基础

神经网络是一种模仿人脑处理信息方式的计算模型，它由许多相互连接的节点（神经元）组成，这些节点按层次排列。

神经网络的强大之处在于其能够自动从大量数据中学习复杂的模式和特征，无需人工设计特征提取器。

随着深度学习的发展，神经网络已经成为解决许多复杂问题的关键技术。

神经元 (Neuron)

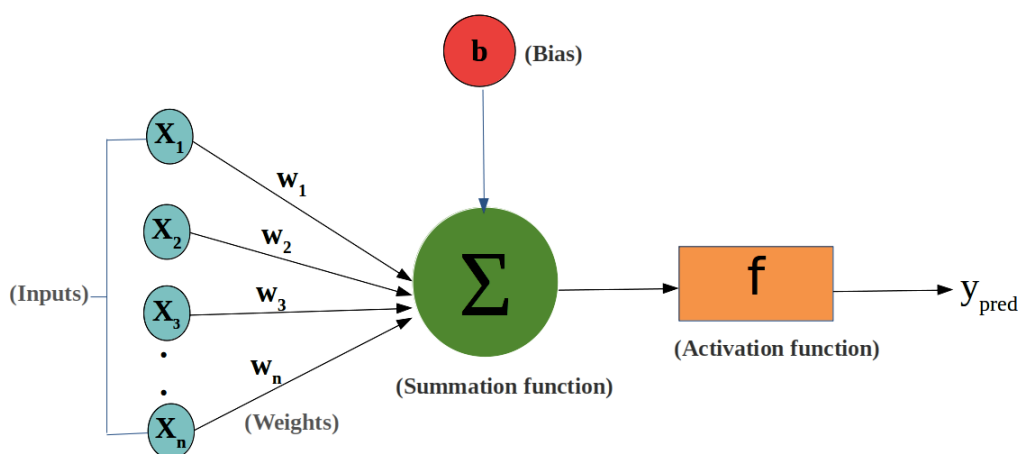
神经元是神经网络的基本单元，它接收输入信号，通过加权求和后与偏置 (bias) 相加，然后通过激活函数处理以产生输出。

神经元的权重和偏置是网络学习过程中需要调整的参数。

输入和输出:

- **输入 (Input)** : 输入是网络的起始点，可以是特征数据，如图像的像素值或文本的词向量。
- **输出 (Output)** : 输出是网络的终点，表示模型的预测结果，如分类任务中的类别标签。

神经元接收多个输入 (例如 $x_1, x_2, \dots, x_n$)，如果输入的加权和大于激活阈值 (activation potential)，则产生二进制输出。



神经元的输出可以看作是输入的加权和加上偏置 (bias)，神经元的数学表示:

$$\text{Output} = \sum_j w_j x_j + \text{Bias}$$

这里， w_j 是权重， x_j 是输入，而 Bias 是偏置项。

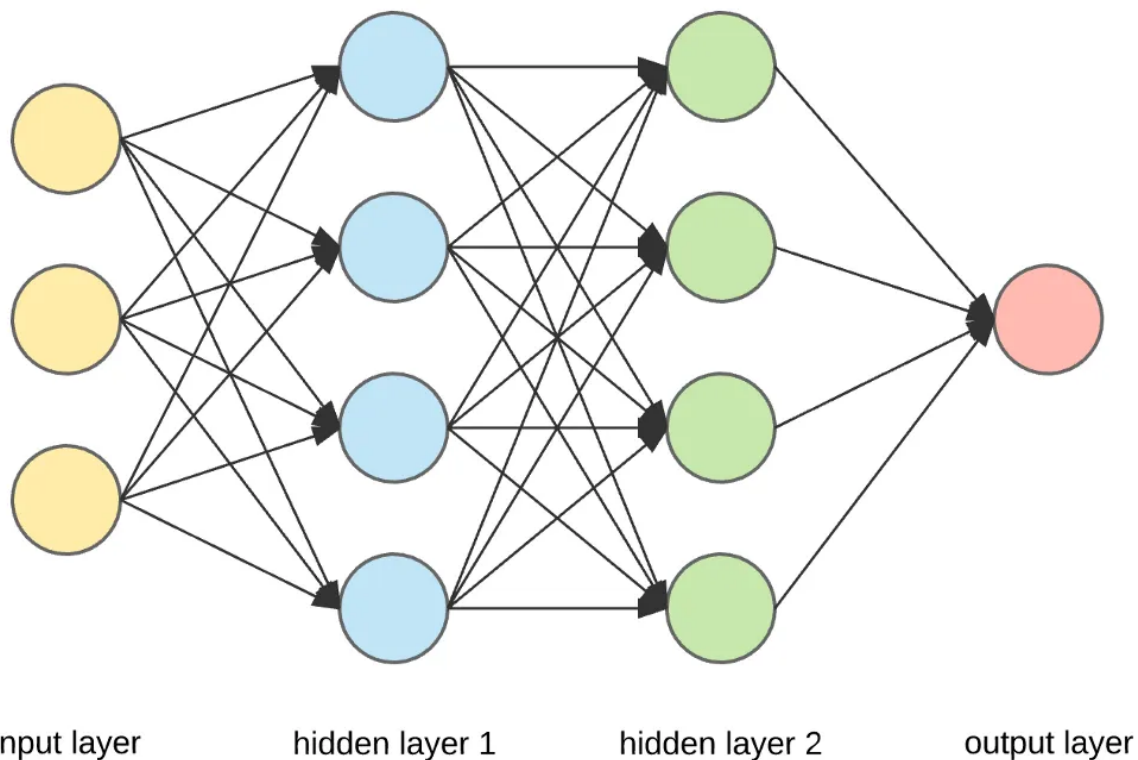
层 (Layer)

输入层和输出层之间的层被称为隐藏层，层与层之间的连接密度和类型构成了网络的配置。

神经网络由多个层组成，包括:

- **输入层 (Input Layer)** : 接收原始输入数据。
- **隐藏层 (Hidden Layer)** : 对输入数据进行处理，可以有多个隐藏层。
- **输出层 (Output Layer)** : 产生最终的输出结果。

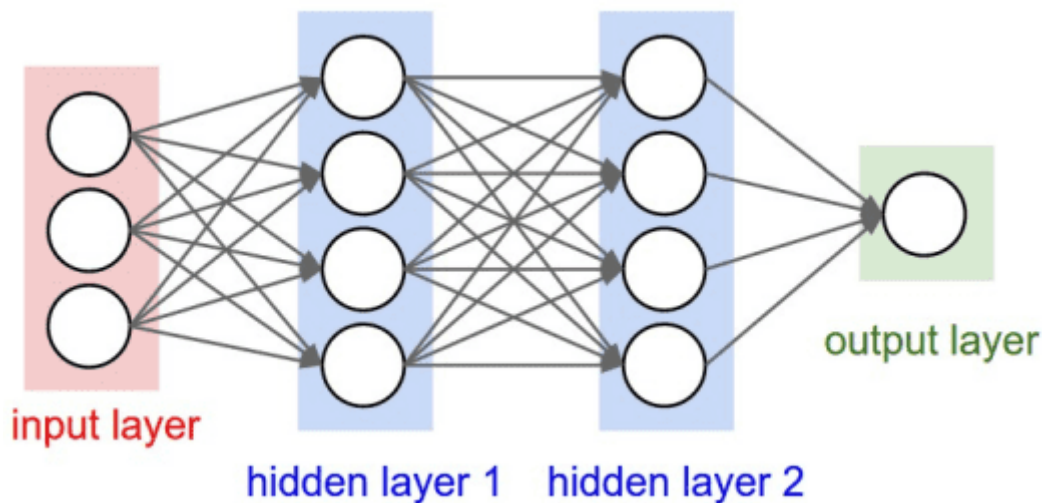
典型的神经网络架构:



前馈神经网络 (Feedforward Neural Network, FNN)

前馈神经网络 (Feedforward Neural Network, FNN) 是神经网络家族中的基本单元。

前馈神经网络特点是数据从输入层开始，经过一个或多个隐藏层，最后到达输出层，全过程没有循环或反馈。



前馈神经网络的基本结构：

- **输入层：** 数据进入网络的入口点。输入层的每个节点代表一个输入特征。
- **隐藏层：** 一个或多个层，用于捕获数据的非线性特征。每个隐藏层由多个神经元组成，每个神经元通过激活函数增加非线性能力。
- **输出层：** 输出网络的预测结果。节点数和问题类型相关，例如分类问题的输出节点数等于类别数。
- **连接权重与偏置：** 每个神经元的输入通过权重进行加权求和，并加上偏置值，然后通过激活函数传递。

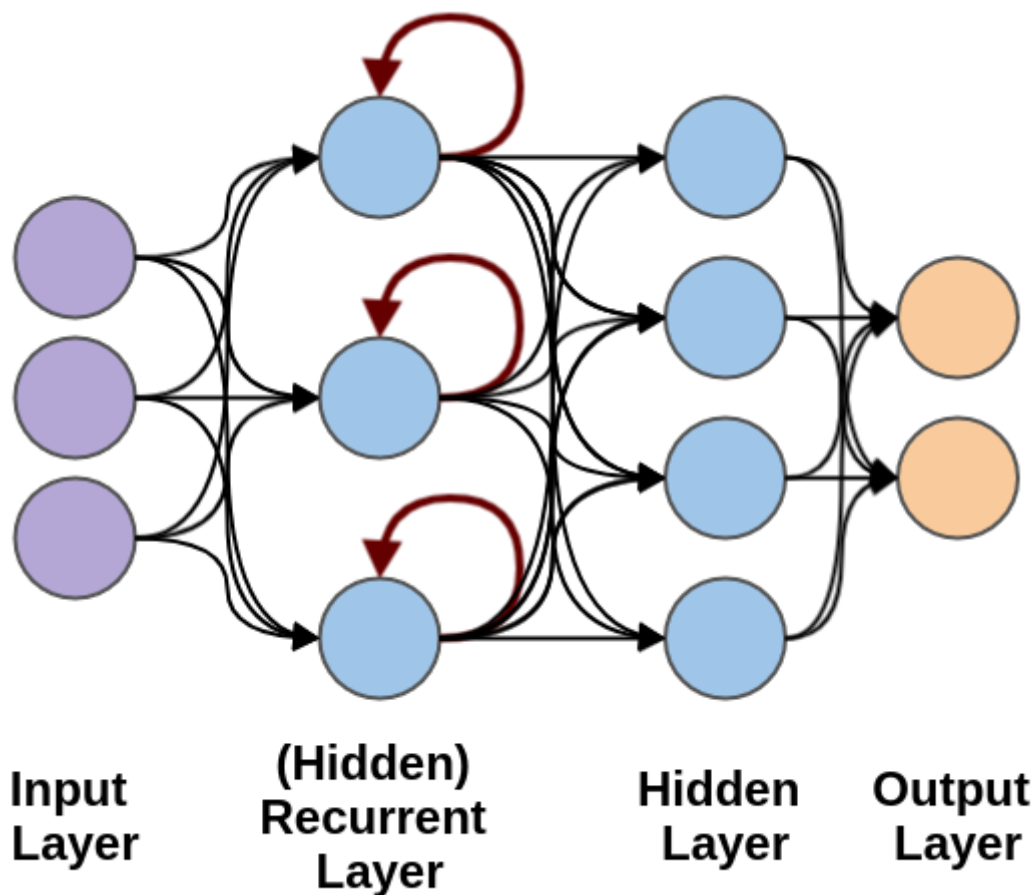
循环神经网络 (Recurrent Neural Network, RNN)

循环神经网络 (Recurrent Neural Network, RNN) 络是一类专门处理序列数据的神经网络，能够捕获输入数据中时间或顺序信息的依赖关系。

RNN 的特别之处在于它具有"记忆能力"，可以在网络的隐藏状态中保存之前时间步的信息。

循环神经网络用于处理随时间变化的数据模式。

在 RNN 中，相同的层被用来接收输入参数，并在指定的神经网络中显示输出参数。



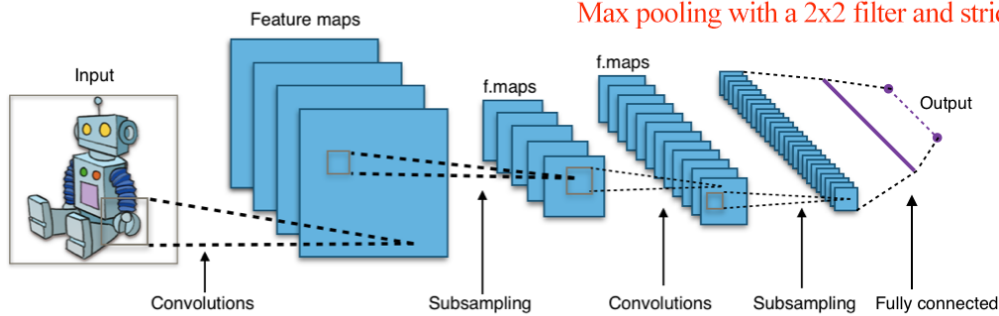
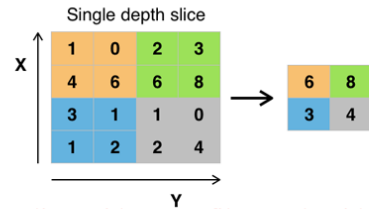
PyTorch 提供了强大的工具来构建和训练神经网络。

神经网络在 PyTorch 中是通过 `torch.nn` 模块来实现的。

`torch.nn` 模块提供了各种网络层（如全连接层、卷积层等）、损失函数和优化器，让神经网络的构建和训练变得更加方便。



CNN



在 PyTorch 中，构建神经网络通常需要继承 `nn.Module` 类。

`nn.Module` 是所有神经网络模块的基类，你需要定义以下两个部分：

- `__init__()`：定义网络层。
- `forward()`：定义数据的前向传播过程。

简单的全连接神经网络（Fully Connected Network）：

```
1 import torch
2 import torch.nn as nn
3
4 # 定义一个简单的神经网络模型
5 class SimpleNN(nn.Module):
6     def __init__(self):
7         super(SimpleNN, self).__init__()
8         # 定义一个输入层到隐藏层的全连接层
9         self.fc1 = nn.Linear(2, 2) # 输入 2 个特征，输出 2 个特征
10        # 定义一个隐藏层到输出层的全连接层
11        self.fc2 = nn.Linear(2, 1) # 输入 2 个特征，输出 1 个预测值
12
13        def forward(self, x):
14            # 前向传播过程
15            x = torch.relu(self.fc1(x)) # 使用 ReLU 激活函数
16            x = self.fc2(x) # 输出层
17            return x
18
19 # 创建模型实例
20 model = SimpleNN()
21
22 # 打印模型
23 print(model)
```

输出结果如下：

```
1 SimpleNN(
2   (fc1): Linear(in_features=2, out_features=2, bias=True)
3   (fc2): Linear(in_features=2, out_features=1, bias=True)
4 )
```


PyTorch 提供了许多常见的神经网络层，以下是几个常见的：

- `nn.Linear(in_features, out_features)`：全连接层，输入 `in_features` 个特征，输出 `out_features` 个特征。
- `nn.Conv2d(in_channels, out_channels, kernel_size)`：2D 卷积层，用于图像处理。
- `nn.MaxPool2d(kernel_size)`：2D 最大池化层，用于降维。
- `nn.ReLU()`：ReLU 激活函数，常用于隐藏层。
- `nn.Softmax(dim)`：Softmax 激活函数，通常用于输出层，适用于多类分类问题。

激活函数 (Activation Function)

激活函数决定了神经元是否应该被激活。它们是非线性函数，使得神经网络能够学习和执行更复杂的任务。常见的激活函数包括：

- Sigmoid：用于二分类问题，输出值在 0 和 1 之间。
- Tanh：输出值在 -1 和 1 之间，常用于输出层之前。
- ReLU (Rectified Linear Unit)：目前最流行的激活函数之一，定义为 $f(x) = \max(0, x)$ ，有助于解决梯度消失问题。
- Softmax：常用于多分类问题的输出层，将输出转换为概率分布。

```
1 import torch.nn.functional as F
2
3 # ReLU 激活
4 output = F.relu(input_tensor)
5
6 # Sigmoid 激活
7 output = torch.sigmoid(input_tensor)
8
9 # Tanh 激活
10 output = torch.tanh(input_tensor)
```

损失函数 (Loss Function)

损失函数用于衡量模型的预测值与真实值之间的差异。

常见的损失函数包括：

- **均方误差 (MSELoss)**：回归问题常用，计算输出与目标值的平方差。
- **交叉熵损失 (CrossEntropyLoss)**：分类问题常用，计算输出和真实标签之间的交叉熵。
- **BCEWithLogitsLoss**：二分类问题，结合了 Sigmoid 激活和二元交叉熵损失。

```
1
2 # 均方误差损失
3 criterion = nn.MSELoss()
4
5 # 交叉熵损失
6 criterion = nn.CrossEntropyLoss()
7
8 # 二分类交叉熵损失
9 criterion = nn.BCEWithLogitsLoss()
```

优化器 (Optimizer)

优化器负责在训练过程中更新网络的权重和偏置。

常见的优化器包括：

- SGD (随机梯度下降)
- Adam (自适应矩估计)
- RMSprop (均方根传播)

```
1
2 import torch.optim as optim
3
4 # 使用 SGD 优化器
5 optimizer = optim.SGD(model.parameters(), lr=0.01)
6
7 # 使用 Adam 优化器
8 optimizer = optim.Adam(model.parameters(), lr=0.001)
```

训练过程 (Training Process)

训练神经网络涉及以下步骤：

1. **准备数据**：通过 `DataLoader` 加载数据。
2. **定义损失函数和优化器**。
3. **前向传播**：计算模型的输出。
4. **计算损失**：与目标进行比较，得到损失值。
5. **反向传播**：通过 `loss.backward()` 计算梯度。
6. **更新参数**：通过 `optimizer.step()` 更新模型的参数。
7. **重复上述步骤**，直到达到预定的训练轮数。

```
1
2 # 假设已经定义好了模型、损失函数和优化器
3
4 # 训练数据示例
5 x = torch.randn(10, 2) # 10 个样本，每个样本有 2 个特征
6 y = torch.randn(10, 1) # 10 个目标标签
7
8 # 训练过程
9 for epoch in range(100): # 训练 100 轮
10     model.train() # 设置模型为训练模式
11     optimizer.zero_grad() # 清除梯度
12     output = model(x) # 前向传播
13     loss = criterion(output, y) # 计算损失
14     loss.backward() # 反向传播
15     optimizer.step() # 更新权重
16
17     if (epoch + 1) % 10 == 0: # 每 10 轮输出一次损失
18         print(f'Epoch [{epoch + 1}/100], Loss: {loss.item():.4f}')
```

测试与评估

训练完成后，需要对模型进行测试和评估。

常见的步骤包括：

- **计算测试集的损失**：测试模型在未见过的数据上的表现。
- **计算准确率 (Accuracy)**：对于分类问题，计算正确预测的比例。

```
1
2 # 假设你有测试集 X_test 和 Y_test
3 model.eval() # 设置模型为评估模式
4 with torch.no_grad(): # 在评估过程中禁用梯度计算
5     output = model(X_test)
6     loss = criterion(output, Y_test)
7     print(f'Test Loss: {loss.item():.4f}')
```

神经网络类型

1. **前馈神经网络 (Feedforward Neural Networks)**：数据单向流动，从输入层到输出层，无反馈连接。
2. **卷积神经网络 (Convolutional Neural Networks, CNNs)**：适用于图像处理，使用卷积层提取空间特征。
3. **循环神经网络 (Recurrent Neural Networks, RNNs)**：适用于序列数据，如时间序列分析和自然语言处理，允许信息反馈循环。
4. **长短期记忆网络 (Long Short-Term Memory, LSTM)**：一种特殊的RNN，能够学习长期依赖关系。

PyTorch 第一个神经网络

本章节我们将介绍如何用 PyTorch 实现一个简单的前馈神经网络，完成一个二分类任务。

以下实例展示了如何使用 PyTorch 实现一个简单的神经网络进行二分类任务训练。

网络结构包括输入层、隐藏层和输出层，使用了 ReLU 激活函数和 Sigmoid 激活函数。

采用了均方误差损失函数和随机梯度下降优化器。

训练过程是通过前向传播、计算损失、反向传播和参数更新来逐步调整模型参数。

```
1 # 导入PyTorch库
2 import torch
3 import torch.nn as nn
4
5 # 定义输入层大小、隐藏层大小、输出层大小和批量大小
6 n_in, n_h, n_out, batch_size = 10, 5, 1, 10
7
8 # 创建虚拟输入数据和目标数据
9 x = torch.randn(batch_size, n_in) # 随机生成输入数据
10 y = torch.tensor([[1.0], [0.0], [0.0],
11                  [1.0], [1.0], [1.0], [0.0], [0.0], [1.0], [1.0]]) # 目标输出数据
12
13 # 创建顺序模型，包含线性层、ReLU激活函数和Sigmoid激活函数
14 model = nn.Sequential(
15     nn.Linear(n_in, n_h), # 输入层到隐藏层的线性变换
```

```

16     nn.ReLU(),          # 隐藏层的ReLU激活函数
17     nn.Linear(n_h, n_out), # 隐藏层到输出层的线性变换
18     nn.Sigmoid()         # 输出层的Sigmoid激活函数
19 )
20
21 # 定义均方误差损失函数和随机梯度下降优化器
22 criterion = torch.nn.MSELoss()
23 optimizer = torch.optim.SGD(model.parameters(), lr=0.01) # 学习率为0.01
24
25 # 执行梯度下降算法进行模型训练
26 for epoch in range(50): # 迭代50次
27     y_pred = model(x) # 前向传播, 计算预测值
28     loss = criterion(y_pred, y) # 计算损失
29     print('epoch: ', epoch, 'loss: ', loss.item()) # 打印损失值
30
31     optimizer.zero_grad() # 清零梯度
32     loss.backward() # 反向传播, 计算梯度
33     optimizer.step() # 更新模型参数

```

输出结果类似如下:

```

1 epoch: 0 loss: 0.2591968774795532
2 epoch: 1 loss: 0.25902628898620605
3 epoch: 2 loss: 0.25885599851608276
4 epoch: 3 loss: 0.25868603587150574
5 epoch: 4 loss: 0.25851646065711975
6 ...

```

定义网络参数:

```

1 n_in, n_h, n_out, batch_size = 10, 5, 1, 10

```

- `n_in`: 输入层大小为 10, 即每个数据点有 10 个特征。
- `n_h`: 隐藏层大小为 5, 即隐藏层包含 5 个神经元。
- `n_out`: 输出层大小为 1, 即输出一个标量, 表示二分类结果 (0 或 1)。
- `batch_size`: 每个批次包含 10 个样本。

生成输入数据和目标数据:

```

1 x = torch.randn(batch_size, n_in) # 随机生成输入数据
2 y = torch.tensor([[1.0], [0.0], [0.0],
3                  [1.0], [1.0], [1.0], [0.0], [0.0], [1.0], [1.0]]) # 目标输出
    数据

```

- `x`: 随机生成一个形状为 `(10, 10)` 的输入数据矩阵, 表示 10 个样本, 每个样本有 10 个特征。
- `y`: 目标输出数据 (标签), 表示每个输入样本的类别标签 (0 或 1), 是一个 `10x1` 的张量。

定义神经网络模型:

```

1 model = nn.Sequential(
2     nn.Linear(n_in, n_h), # 输入层到隐藏层的线性变换
3     nn.ReLU(),           # 隐藏层的ReLU激活函数
4     nn.Linear(n_h, n_out), # 隐藏层到输出层的线性变换
5     nn.Sigmoid()         # 输出层的Sigmoid激活函数
6 )

```

`nn.Sequential` 用于按顺序定义网络层。

- `nn.Linear(n_in, n_h)`: 定义输入层到隐藏层的线性变换，输入特征是 10 个，隐藏层有 5 个神经元。
- `nn.ReLU()`: 在隐藏层后添加 ReLU 激活函数，增加非线性。
- `nn.Linear(n_h, n_out)`: 定义隐藏层到输出层的线性变换，输出为 1 个神经元。
- `nn.Sigmoid()`: 输出层使用 Sigmoid 激活函数，将结果映射到 0 到 1 之间，用于二分类任务。

定义损失函数和优化器：

```

1 criterion = torch.nn.MSELoss() # 使用均方误差损失函数
2 optimizer = torch.optim.SGD(model.parameters(), lr=0.01) # 使用随机梯度下降优化器，学习率为 0.01

```

训练循环：

```

1 for epoch in range(50): # 训练50轮
2     y_pred = model(x) # 前向传播，计算预测值
3     loss = criterion(y_pred, y) # 计算损失
4     print('epoch: ', epoch, 'loss: ', loss.item()) # 打印损失值
5
6     optimizer.zero_grad() # 清零梯度
7     loss.backward() # 反向传播，计算梯度
8     optimizer.step() # 更新模型参数

```

- `for epoch in range(50)`: 进行 50 次训练迭代。
- `y_pred = model(x)`: 进行前向传播，使用当前模型参数计算输入数据 `x` 的预测值。
- `loss = criterion(y_pred, y)`: 计算预测值和目标值 `y` 之间的损失。
- `optimizer.zero_grad()`: 清除上一轮训练时的梯度值。
- `loss.backward()`: 反向传播，计算损失函数相对于模型参数的梯度。
- `optimizer.step()`: 根据计算出的梯度更新模型参数。

可视化代码：

```

1
2 import torch
3 import torch.nn as nn
4 import matplotlib.pyplot as plt
5
6 # 定义输入层大小、隐藏层大小、输出层大小和批量大小
7 n_in, n_h, n_out, batch_size = 10, 5, 1, 10
8
9 # 创建虚拟输入数据和目标数据
10 x = torch.randn(batch_size, n_in) # 随机生成输入数据
11 y = torch.tensor([[1.0], [0.0], [0.0],
12                  [1.0], [1.0], [1.0], [0.0], [0.0], [1.0], [1.0]]) # 目标输出数据

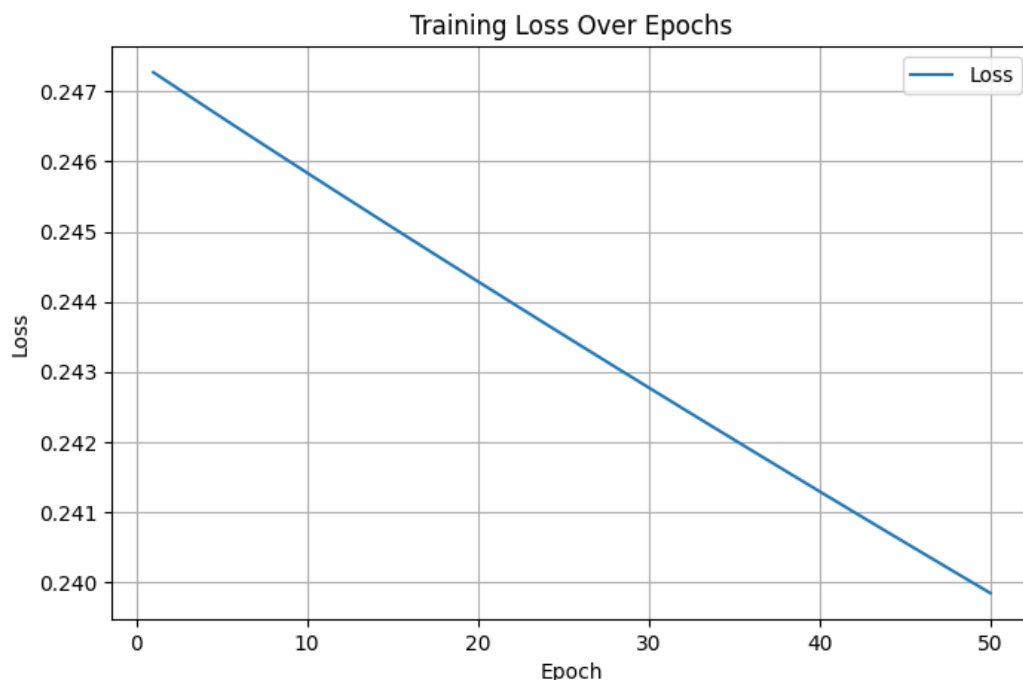
```

```

13
14 # 创建顺序模型，包含线性层、ReLU激活函数和Sigmoid激活函数
15 model = nn.Sequential(
16     nn.Linear(n_in, n_h), # 输入层到隐藏层的线性变换
17     nn.ReLU(),            # 隐藏层的ReLU激活函数
18     nn.Linear(n_h, n_out), # 隐藏层到输出层的线性变换
19     nn.Sigmoid()          # 输出层的Sigmoid激活函数
20 )
21
22 # 定义均方误差损失函数和随机梯度下降优化器
23 criterion = torch.nn.MSELoss()
24 optimizer = torch.optim.SGD(model.parameters(), lr=0.01) # 学习率为0.01
25
26 # 用于存储每轮的损失值
27 losses = []
28
29 # 执行梯度下降算法进行模型训练
30 for epoch in range(50): # 迭代50次
31     y_pred = model(x) # 前向传播，计算预测值
32     loss = criterion(y_pred, y) # 计算损失
33     losses.append(loss.item()) # 记录损失值
34     print(f'Epoch [{epoch+1}/50], Loss: {loss.item():.4f}') # 打印损失值
35
36     optimizer.zero_grad() # 清零梯度
37     loss.backward() # 反向传播，计算梯度
38     optimizer.step() # 更新模型参数
39
40 # 可视化损失变化曲线
41 plt.figure(figsize=(8, 5))
42 plt.plot(range(1, 51), losses, label='Loss')
43 plt.xlabel('Epoch')
44 plt.ylabel('Loss')
45 plt.title('Training Loss Over Epochs')
46 plt.legend()
47 plt.grid()
48 plt.show()
49
50 # 可视化预测结果与实际目标值对比
51 y_pred_final = model(x).detach().numpy() # 最终预测值
52 y_actual = y.numpy() # 实际值
53
54 plt.figure(figsize=(8, 5))
55 plt.plot(range(1, batch_size + 1), y_actual, 'o-', label='Actual',
56         color='blue')
57 plt.plot(range(1, batch_size + 1), y_pred_final, 'x--', label='Predicted',
58         color='red')
59 plt.xlabel('Sample Index')
60 plt.ylabel('Value')
61 plt.title('Actual vs Predicted Values')
62 plt.legend()
63 plt.grid()
64 plt.show()

```

显示如下所示：



另外一个实例

我们假设有一个二维数据集，目标是根据点的位置将它们分类到两个类别中（例如，红色和蓝色点）。

以下实例展示了如何使用神经网络完成简单的二分类任务，为更复杂的任务奠定了基础，通过 PyTorch 的模块化接口，神经网络的构建、训练和可视化都非常直观。

1、数据准备

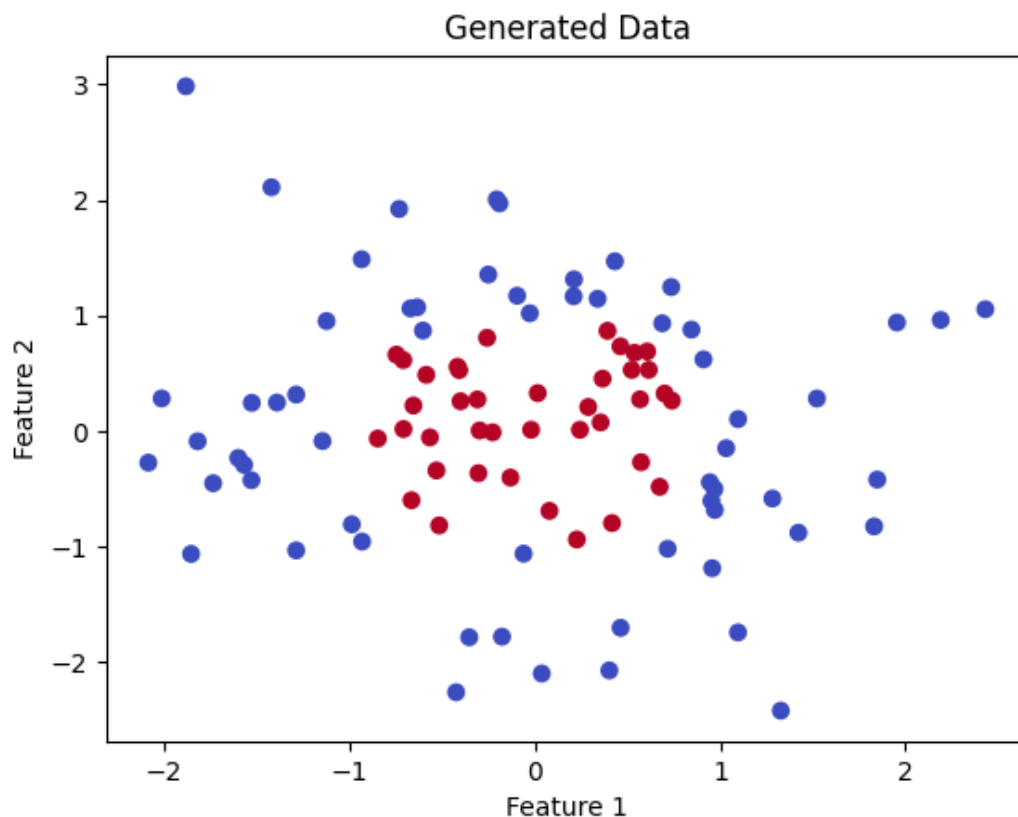
首先，我们生成一些简单的二维数据：

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import matplotlib.pyplot as plt
5
6 # 生成一些随机数据
7 n_samples = 100
8 data = torch.randn(n_samples, 2) # 生成 100 个二维数据点
9 labels = (data[:, 0]**2 + data[:, 1]**2 < 1).float().unsqueeze(1) # 点在圆内
   为1，圆外为0
10
11 # 可视化数据
12 plt.scatter(data[:, 0], data[:, 1], c=labels.squeeze(), cmap='coolwarm')
13 plt.title("Generated Data")
14 plt.xlabel("Feature 1")
15 plt.ylabel("Feature 2")
16 plt.show()
```

数据说明：

- `data` 是输入的二维点，每个点有两个特征。
- `labels` 是目标分类，点在圆形区域内为 1，否则为 0。

显示如下：



2、定义神经网络

用 PyTorch 创建一个简单的前馈神经网络。

前馈神经网络使用了一层隐藏层，通过简单的线性变换和激活函数捕获数据的非线性模式。

```
1 class SimpleNN(nn.Module):
2     def __init__(self):
3         super(SimpleNN, self).__init__()
4         # 定义神经网络的层
5         self.fc1 = nn.Linear(2, 4) # 输入层有 2 个特征，隐藏层有 4 个神经元
6         self.fc2 = nn.Linear(4, 1) # 隐藏层输出到 1 个神经元（用于二分类）
7         self.sigmoid = nn.Sigmoid() # 二分类激活函数
8
9     def forward(self, x):
10        x = torch.relu(self.fc1(x)) # 使用 ReLU 激活函数
11        x = self.sigmoid(self.fc2(x)) # 输出层使用 Sigmoid 激活函数
12        return x
```

实例化模型

```
model = SimpleNN()
```

3、定义损失函数和优化器

```
1 # 定义二分类的损失函数和优化器
2 criterion = nn.BCELoss() # 二元交叉熵损失
3 optimizer = optim.SGD(model.parameters(), lr=0.1) # 使用随机梯度下降优化器
```


4、训练模型

用数据训练模型，让它学会分类。

```
1 # 训练
2 epochs = 100
3 for epoch in range(epochs):
4     # 前向传播
5     outputs = model(data)
6     loss = criterion(outputs, labels)
7
8     # 反向传播
9     optimizer.zero_grad()
10    loss.backward()
11    optimizer.step()
12
13    # 每 10 轮打印一次损失
14    if (epoch + 1) % 10 == 0:
15        print(f'Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.4f}')
```

5、测试模型并可视化结果

我们测试模型，并在图像上绘制决策边界。

```
1 # 可视化决策边界
2 def plot_decision_boundary(model, data):
3     x_min, x_max = data[:, 0].min() - 1, data[:, 0].max() + 1
4     y_min, y_max = data[:, 1].min() - 1, data[:, 1].max() + 1
5     xx, yy = torch.meshgrid(torch.arange(x_min, x_max, 0.1),
6                               torch.arange(y_min, y_max, 0.1), indexing='ij')
7     grid = torch.cat([xx.reshape(-1, 1), yy.reshape(-1, 1)], dim=1)
8     predictions = model(grid).detach().numpy().reshape(xx.shape)
9     plt.contourf(xx, yy, predictions, levels=[0, 0.5, 1], cmap='coolwarm',
10                alpha=0.7)
11    plt.scatter(data[:, 0], data[:, 1], c=labels.squeeze(), cmap='coolwarm',
12               edgecolors='k')
13    plt.title("Decision Boundary")
14    plt.show()
15 plot_decision_boundary(model, data)
```

6、完整代码

完整代码如下：

```
1
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import matplotlib.pyplot as plt
6
7 # 生成一些随机数据
8 n_samples = 100
9 data = torch.randn(n_samples, 2) # 生成 100 个二维数据点
```

```

10 labels = (data[:, 0]**2 + data[:, 1]**2 < 1).float().unsqueeze(1) # 点在圆内
    为1, 圆外为0
11
12 # 可视化数据
13 plt.scatter(data[:, 0], data[:, 1], c=labels.squeeze(), cmap='coolwarm')
14 plt.title("Generated Data")
15 plt.xlabel("Feature 1")
16 plt.ylabel("Feature 2")
17 plt.show()
18
19 # 定义前馈神经网络
20 class SimpleNN(nn.Module):
21     def __init__(self):
22         super(SimpleNN, self).__init__()
23         # 定义神经网络的层
24         self.fc1 = nn.Linear(2, 4) # 输入层有 2 个特征, 隐藏层有 4 个神经元
25         self.fc2 = nn.Linear(4, 1) # 隐藏层输出到 1 个神经元 (用于二分类)
26         self.sigmoid = nn.Sigmoid() # 二分类激活函数
27
28     def forward(self, x):
29         x = torch.relu(self.fc1(x)) # 使用 ReLU 激活函数
30         x = self.sigmoid(self.fc2(x)) # 输出层使用 Sigmoid 激活函数
31         return x
32
33 # 实例化模型
34 model = SimpleNN()
35
36 # 定义损失函数和优化器
37 criterion = nn.BCELoss() # 二元交叉熵损失
38 optimizer = optim.SGD(model.parameters(), lr=0.1) # 使用随机梯度下降优化器
39
40 # 训练
41 epochs = 100
42 for epoch in range(epochs):
43     # 前向传播
44     outputs = model(data)
45     loss = criterion(outputs, labels)
46
47     # 反向传播
48     optimizer.zero_grad()
49     loss.backward()
50     optimizer.step()
51
52     # 每 10 轮打印一次损失
53     if (epoch + 1) % 10 == 0:
54         print(f'Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.4f}')
55
56 # 可视化决策边界
57 def plot_decision_boundary(model, data):
58     x_min, x_max = data[:, 0].min() - 1, data[:, 0].max() + 1
59     y_min, y_max = data[:, 1].min() - 1, data[:, 1].max() + 1
60     xx, yy = torch.meshgrid(torch.arange(x_min, x_max, 0.1),
61                             torch.arange(y_min, y_max, 0.1), indexing='ij')
62     grid = torch.cat([xx.reshape(-1, 1), yy.reshape(-1, 1)], dim=1)
63     predictions = model(grid).detach().numpy().reshape(xx.shape)
64     plt.contourf(xx, yy, predictions, levels=[0, 0.5, 1], cmap='coolwarm',
65                 alpha=0.7)

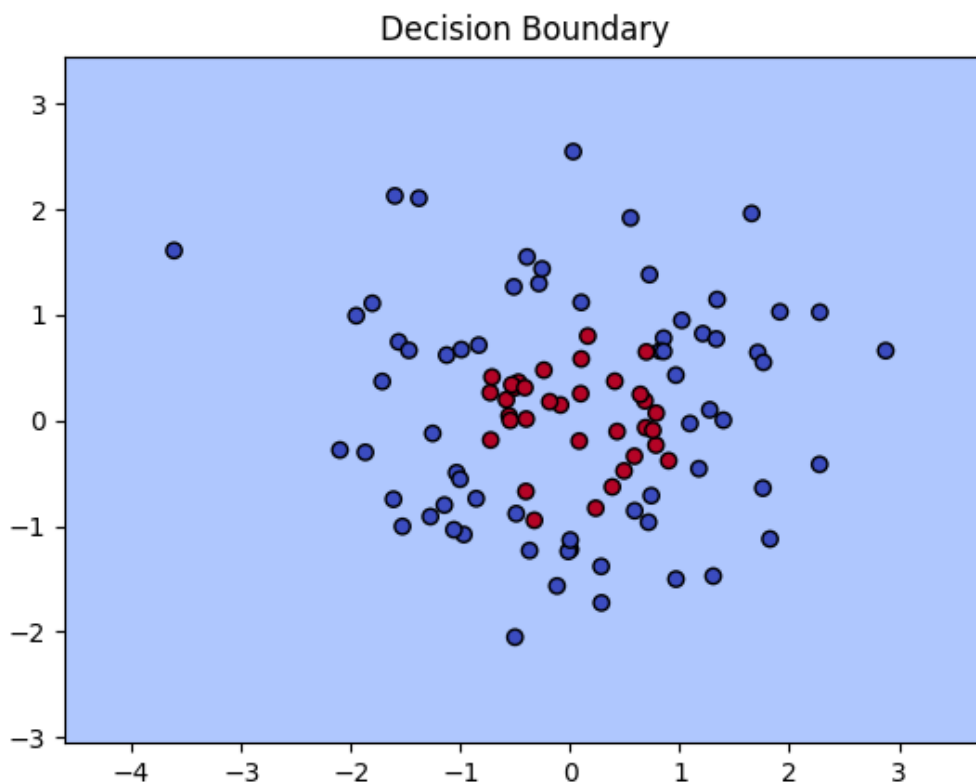
```

```
64 plt.scatter(data[:, 0], data[:, 1], c=labels.squeeze(), cmap='coolwarm',  
edgecolors='k')  
65 plt.title("Decision Boundary")  
66 plt.show()  
67  
68 plot_decision_boundary(model, data)
```

训练时的损失输出：

```
1 Epoch [10/100], Loss: 0.5247  
2 Epoch [20/100], Loss: 0.3142  
3 ...  
4 Epoch [100/100], Loss: 0.0957
```

图中显示了原始数据点（红色和蓝色），以及模型学习到的分类边界。



PyTorch 模型保存和加载

在深度学习项目中，模型保存和加载是至关重要的环节，主要原因包括：

1. **训练中断恢复**：当训练过程意外中断时，可以从保存点继续训练
2. **模型部署**：将训练好的模型部署到生产环境
3. **模型共享**：方便团队成员之间共享模型成果
4. **迁移学习**：保存预训练模型用于其他任务
5. **性能评估**：保存不同训练阶段的模型进行比较

基本保存和加载方法

保存整个模型

这是最简单的方法，保存模型的架构和参数：

```
1
2 import torch
3 import torchvision.models as models
4
5 # 创建并训练一个模型
6 model = models.resnet18(pretrained=True)
7 # ... 训练代码 ...
8
9 # 保存整个模型
10 torch.save(model, 'model.pth')
11
12 # 加载整个模型
13 loaded_model = torch.load('model.pth')
```

优点：

- 代码简单直观
- 保存了完整的模型结构

缺点：

- 文件体积较大
- 对模型类的定义有依赖

仅保存模型参数（推荐方式）

更推荐的方式是只保存模型的状态字典(state_dict)：

```
1
2 # 保存模型参数
3 torch.save(model.state_dict(), 'model_weights.pth')
4
5 # 加载模型参数
6 model = models.resnet18() # 必须先创建相同架构的模型
7 model.load_state_dict(torch.load('model_weights.pth'))
8 model.eval() # 设置为评估模式
```

优点：

- 文件更小
- 更灵活，可以加载到不同架构中
- 兼容性更好

保存和加载训练状态

在实际项目中，我们通常还需要保存优化器状态、epoch等信息：

```
1 # 保存检查点
2 checkpoint = {
3     'epoch': epoch,
4     'model_state_dict': model.state_dict(),
```

```
5     'optimizer_state_dict': optimizer.state_dict(),
6     'loss': loss,
7     # 可以添加其他需要保存的信息
8 }
9
10 torch.save(checkpoint, 'checkpoint.pth')
11
12 # 加载检查点
13 checkpoint = torch.load('checkpoint.pth')
14 model.load_state_dict(checkpoint['model_state_dict'])
15 optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
16 epoch = checkpoint['epoch']
17 loss = checkpoint['loss']
18
19 model.eval() # 或者 model.train() 取决于你的需求
```

跨设备加载模型

CPU/GPU兼容性处理

```
1 # 保存时指定map_location
2 torch.save(model.state_dict(), 'model_weights.pth')
3
4 # 加载到CPU（当模型是在GPU上训练时）
5 device = torch.device('cpu')
6 model.load_state_dict(torch.load('model_weights.pth', map_location=device))
7
8 # 加载到GPU
9 device = torch.device('cuda')
10 model.load_state_dict(torch.load('model_weights.pth', map_location=device))
11
11 model.to(device)
```

多GPU训练模型加载

```
1 # 保存多GPU模型
2 torch.save(model.module.state_dict(), 'multigpu_model.pth')
3
4 # 加载到单GPU
5 model = ModelClass()
6 model.load_state_dict(torch.load('multigpu_model.pth'))
```

模型转换与兼容性

PyTorch版本兼容性

```
1
2 # 保存时指定_use_new_zipfile_serialization=True以获得更好的兼容性
3 torch.save(model.state_dict(), 'model.pth',
4             _use_new_zipfile_serialization=True)
```

转换为TorchScript

```
1 # 将模型转换为TorchScript格式
2 scripted_model = torch.jit.script(model)
3 torch.jit.save(scripted_model, 'model_scripted.pt')
4
5 # 加载TorchScript模型
6 loaded_script = torch.jit.load('model_scripted.pt')
```

最佳实践与常见问题

最佳实践

1. **命名规范**: 使用有意义的文件名, 如 `resnet18_epoch50.pth`
2. **定期保存**: 每隔几个epoch保存一次检查点
3. **验证加载**: 保存后立即测试加载功能
4. **文档记录**: 记录模型架构和训练参数
5. **版本控制**: 将模型文件纳入版本控制系统

常见问题解决方案

问题1: Missing key(s) in state_dict

解决: 确保模型架构完全匹配, 或使用 `strict=False` 参数:

```
1 model.load_state_dict(torch.load('model.pth'), strict=False)
```

问题2: CUDA out of memory

解决: 加载时先放到CPU:

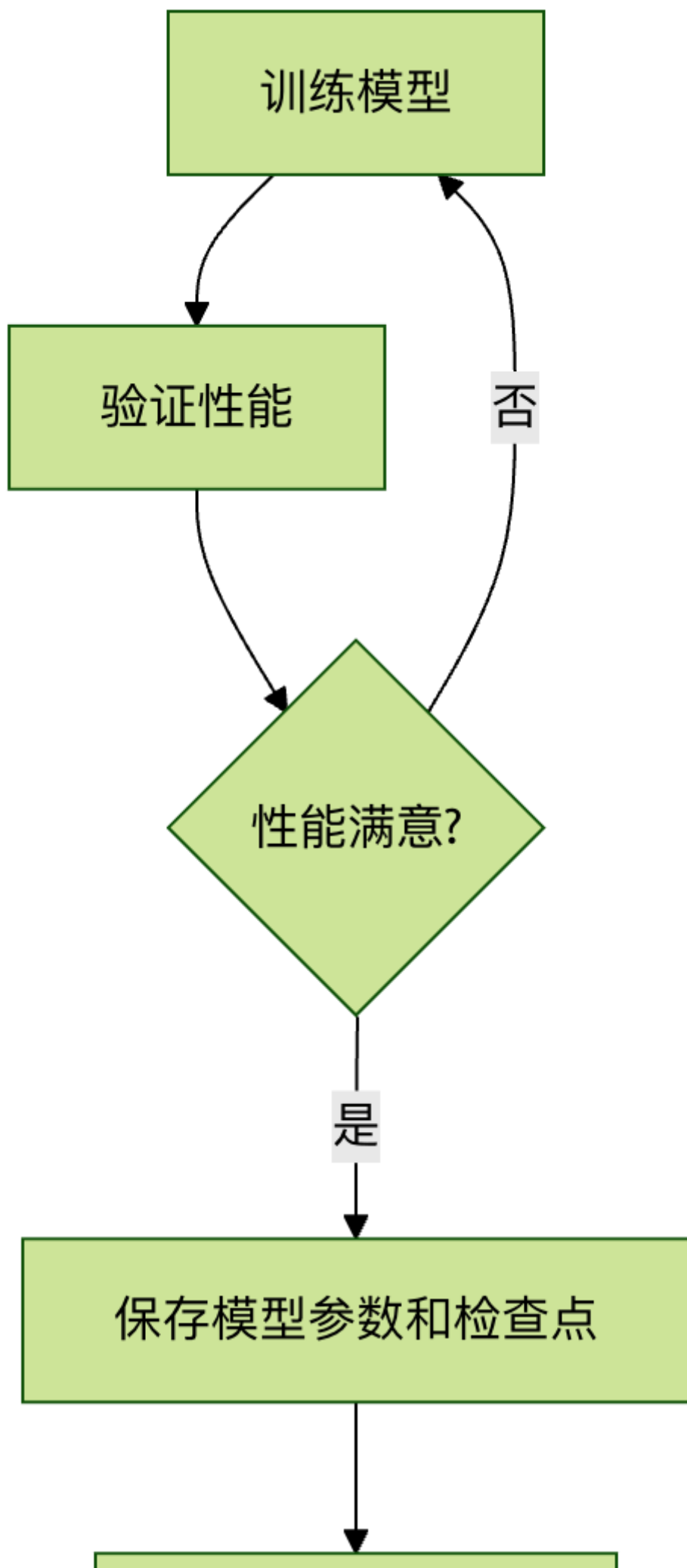
```
1
2 model.load_state_dict(torch.load('model.pth', map_location='cpu'))
```

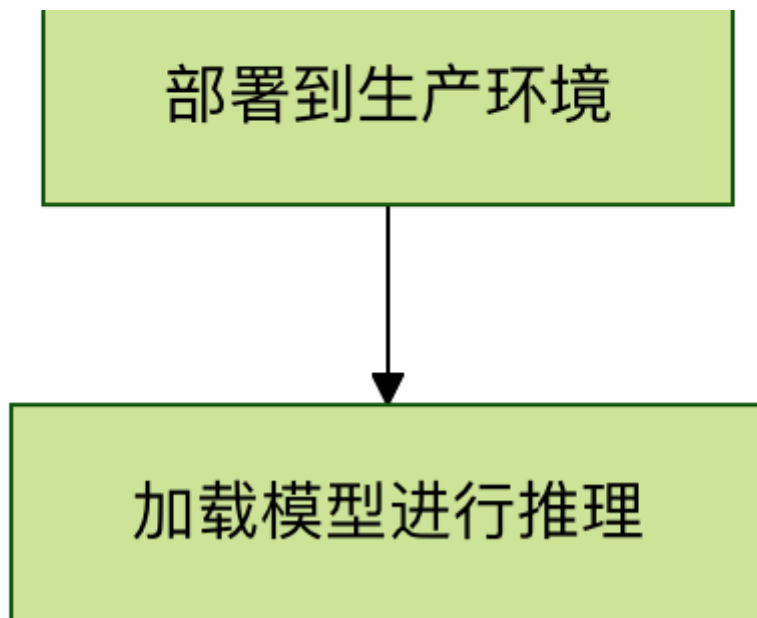
问题3: 无法加载旧版本模型

解决: 尝试在不同PyTorch版本中加载, 或转换模型格式

实际应用示例

图像分类模型保存与加载流程





完整代码示例

```
1
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5
6 # 定义一个简单模型
7 class SimpleModel(nn.Module):
8     def __init__(self):
9         super(SimpleModel, self).__init__()
10        self.fc = nn.Linear(10, 2)
11
12        def forward(self, x):
13            return self.fc(x)
14
15 # 初始化
16 model = SimpleModel()
17 optimizer = optim.SGD(model.parameters(), lr=0.01)
18 criterion = nn.CrossEntropyLoss()
19
20 # 模拟训练过程
21 for epoch in range(5):
22     # 模拟训练步骤
23     inputs = torch.randn(32, 10)
24     labels = torch.randint(0, 2, (32,))
25
26     optimizer.zero_grad()
27     outputs = model(inputs)
28     loss = criterion(outputs, labels)
29     loss.backward()
30     optimizer.step()
31
32     # 每2个epoch保存一次检查点
33     if epoch % 2 == 0:
34         checkpoint = {
35             'epoch': epoch,
36             'model_state_dict': model.state_dict(),
```



```
37         'optimizer_state_dict': optimizer.state_dict(),
38         'loss': loss.item(),
39     }
40     torch.save(checkpoint, f'checkpoint_epoch{epoch}.pth')
41     print(f'Checkpoint saved at epoch {epoch}')
42
43 # 最终保存
44 torch.save(model.state_dict(), 'final_model.pth')
45
46 # 加载示例
47 loaded_model = SimpleModel()
48 loaded_model.load_state_dict(torch.load('final_model.pth'))
49 loaded_model.eval()
50
51 # 测试加载的模型
52 test_input = torch.randn(1, 10)
53 with torch.no_grad():
54     output = loaded_model(test_input)
55     print(f'Test output: {output}')
```

Pytorch torch 参考手册

PyTorch 软件包包含了用于多维张量的数据结构，并定义了在这些张量上执行的数学运算。此外，它还提供了许多实用工具，用于高效地序列化张量和任意类型的数据，以及其他有用的工具。

它还有一个 CUDA 版本，可以让你在计算能力 ≥ 3.0 的 NVIDIA GPU 上运行张量计算。

PyTorch torch API 手册

类别	API	描述
Tensors	<code>[is_tensor(obj)]</code> (https://www.runoob.com/pytorch/pytorch-is_tensor.html)	检查是否为 Py 量。
	<code>is_storage(obj)</code>	检查是否为 Py 对象。
	<code>is_complex(input)</code>	检查数据类型是否为复数。
	<code>is_conj(input)</code>	检查是否为共轭。
	<code>is_floating_point(input)</code>	检查数据类型是否为浮点数据。
	<code>is_nonzero(input)</code>	检查是否为非零张量。
	<code>set_default_dtype(d)</code>	设置默认数据类型。

类别	API	描述
	<code>get_default_dtype()</code>	获取当前默认的数据类型。 <code>torch</code>
	<code>set_default_device(device)</code>	设置默认的设备。 <code>torch</code> 分配的。 <code>device</code>
	<code>get_default_device()</code>	获取默认的设备。 <code>torch</code> 分配的。
	<code>numel(input)</code>	返回输入张量中的元素数量。
Creation Ops	<code>tensor(data)</code>	通过复制构造并返回张量。 历史的。
	<code>sparse_coo_tensor(indices, values)</code>	在指定索引处构造稀疏张量。 <code>indices</code> 构造稀疏张量。 有指定值的。
	<code>as_tensor(data)</code>	将数据转换为张量。 张量， 并尽可能保留梯度。
	<code>zeros(size)</code>	返回一个值为 0 的张量，并指定大小。 <code>size</code>
	<code>ones(size)</code>	返回一个值为 1 的张量，并指定大小。 <code>size</code>
	<code>arange(start, end, step)</code>	返回一个张量，包含从 <code>start</code> 到 <code>end</code> 的值，步长为 <code>step</code> 。 <code>start</code> 的值， <code>step</code>
	<code>rand(size)</code>	返回一个在 (0, 1) 区间内随机分布的张量。

类别	API	描述
	<code>randn(size)</code>	返回一个正态分布张量。
Math operations	<code>add(input, other, alpha)</code>	将 <code>other</code> 乘以 <code>alpha</code> 后加到 <code>input</code> 上。
	<code>mul(input, other)</code>	将 <code>input</code> 与 <code>other</code> 相乘。
	<code>matmul(input, other)</code>	执行矩阵乘法。
	<code>mean(input, dim)</code>	计算 <code>input</code> 在 <code>dim</code> 维度上的均值。
	<code>sum(input, dim)</code>	计算 <code>input</code> 在 <code>dim</code> 维度上的和。
	<code>max(input, dim)</code>	返回 <code>input</code> 在 <code>dim</code> 维度上的最大值。
	<code>min(input, dim)</code>	返回 <code>input</code> 在 <code>dim</code> 维度上的最小值。

Tensor 创建

函数	描述
<code>torch.tensor(data, dtype, device, requires_grad)</code>	从数据创建张量。
<code>torch.as_tensor(data, dtype, device)</code>	将数据转换为张量（共享内存）。
<code>torch.from_numpy(ndarray)</code>	从 NumPy 数组创建张量（共享内存）。
<code>torch.zeros(*size, dtype, device, requires_grad)</code>	创建全零张量。
<code>torch.ones(*size, dtype, device, requires_grad)</code>	创建全一张量。
<code>torch.empty(*size, dtype, device, requires_grad)</code>	创建未初始化的张量。
<code>torch.arange(start, end, step, dtype, device, requires_grad)</code>	创建等差序列张量。
<code>torch.linspace(start, end, steps, dtype, device, requires_grad)</code>	创建等间隔序列张量。
<code>torch.logspace(start, end, steps, base, dtype, device, requires_grad)</code>	创建对数间隔序列张量。
<code>torch.eye(n, m, dtype, device, requires_grad)</code>	创建单位矩阵。
<code>torch.full(size, fill_value, dtype, device, requires_grad)</code>	创建填充指定值的张量。
<code>torch.rand(*size, dtype, device, requires_grad)</code>	创建均匀分布随机张量（范围 $[0, 1)$ ）。
<code>torch.randn(*size, dtype, device, requires_grad)</code>	创建标准正态分布随机张量。
<code>torch.randint(low, high, size, dtype, device, requires_grad)</code>	创建整数随机张量。
<code>torch.randperm(n, dtype, device, requires_grad)</code>	创建 0 到 $n-1$ 的随机排列。

Tensor 操作

函数	描述
<code>torch.cat(tensors, dim)</code>	沿指定维度连接张量。
<code>torch.stack(tensors, dim)</code>	沿新维度堆叠张量。
<code>torch.split(tensor, split_size, dim)</code>	将张量沿指定维度分割。
<code>torch.chunk(tensor, chunks, dim)</code>	将张量沿指定维度分块。
<code>torch.reshape(input, shape)</code>	改变张量的形状。
<code>torch.transpose(input, dim0, dim1)</code>	交换张量的两个维度。
<code>torch.squeeze(input, dim)</code>	移除大小为 1 的维度。
<code>torch.unsqueeze(input, dim)</code>	在指定位置插入大小为 1 的维度。
<code>torch.expand(input, size)</code>	扩展张量的尺寸。
<code>torch.narrow(input, dim, start, length)</code>	返回张量的切片。
<code>torch.permute(input, dims)</code>	重新排列张量的维度。
<code>torch.masked_select(input, mask)</code>	根据布尔掩码选择元素。
<code>torch.index_select(input, dim, index)</code>	沿指定维度选择索引对应的元素。
<code>torch.gather(input, dim, index)</code>	沿指定维度收集指定索引的元素。
<code>torch.scatter(input, dim, index, src)</code>	将 <code>src</code> 的值散布到 <code>input</code> 的指定位置。
<code>torch.nonzero(input)</code>	返回非零元素的索引。

数学运算

函数	描述
<code>torch.add(input, other)</code>	逐元素加法。
<code>torch.sub(input, other)</code>	逐元素减法。
<code>torch.mul(input, other)</code>	逐元素乘法。
<code>torch.div(input, other)</code>	逐元素除法。
<code>torch.matmul(input, other)</code>	矩阵乘法。
<code>torch.pow(input, exponent)</code>	逐元素幂运算。
<code>torch.sqrt(input)</code>	逐元素平方根。
<code>torch.exp(input)</code>	逐元素指数函数。
<code>torch.log(input)</code>	逐元素自然对数。
<code>torch.sum(input, dim)</code>	沿指定维度求和。
<code>torch.mean(input, dim)</code>	沿指定维度求均值。
<code>torch.max(input, dim)</code>	沿指定维度求最大值。
<code>torch.min(input, dim)</code>	沿指定维度求最小值。
<code>torch.abs(input)</code>	逐元素绝对值。
<code>torch.clamp(input, min, max)</code>	将张量值限制在指定范围内。
<code>torch.round(input)</code>	逐元素四舍五入。
<code>torch.floor(input)</code>	逐元素向下取整。
<code>torch.ceil(input)</code>	逐元素向上取整。

随机数生成

函数	描述
<code>torch.manual_seed(seed)</code>	设置随机种子。
<code>torch.initial_seed()</code>	返回当前随机种子。
<code>torch.rand(*size)</code>	创建均匀分布随机张量（范围 [0, 1)）。
<code>torch.randn(*size)</code>	创建标准正态分布随机张量。
<code>torch.randint(low, high, size)</code>	创建整数随机张量。
<code>torch.randperm(n)</code>	返回 0 到 n-1 的随机排列。

线性代数

函数	描述
<code>torch.dot(input, other)</code>	计算两个向量的点积。
<code>torch.mm(input, mat2)</code>	矩阵乘法。
<code>torch.bmm(input, mat2)</code>	批量矩阵乘法。
<code>torch.eig(input)</code>	计算矩阵的特征值和特征向量。
<code>torch.svd(input)</code>	计算矩阵的奇异值分解。
<code>torch.inverse(input)</code>	计算矩阵的逆。
<code>torch.det(input)</code>	计算矩阵的行列式。
<code>torch.trace(input)</code>	计算矩阵的迹。

设备管理

函数	描述
<code>torch.cuda.is_available()</code>	检查 CUDA 是否可用。
<code>torch.device(device)</code>	创建一个设备对象（如 <code>'cpu'</code> 或 <code>'cuda:0'</code> ）。
<code>torch.to(device)</code>	将张量移动到指定设备。

#

```
1
2 import torch
3
4 # 创建张量
5 x = torch.tensor([1, 2, 3])
6 y = torch.zeros(2, 3)
7
8 # 数学运算
9 z = torch.add(x, 1) # 逐元素加 1
10 print(z)
11
12 # 索引和切片
13 mask = x > 1
14 selected = torch.masked_select(x, mask)
15 print(selected)
16
17 # 设备管理
18 if torch.cuda.is_available():
19     device = torch.device('cuda')
20     x = x.to(device)
21     print(x.device)
```


如果需要更详细的信息，可以参考 [PyTorch 官方文档](#)。

PyTorch torch.nn 参考手册

PyTorch 的 `torch.nn` 模块是构建和训练神经网络的核心模块，它提供了丰富的类和函数来定义和操作神经网络。

以下是 `torch.nn` 模块的一些关键组成部分及其功能：

1、nn.Module 类：

- `nn.Module` 是所有自定义神经网络模型的基类。用户通常会从这个类派生自己的模型类，并在其中定义网络层结构以及前向传播函数（forward pass）。

2、预定义层（Modules）：

- 包括各种类型的层组件，例如卷积层（`nn.Conv1d`, `nn.Conv2d`, `nn.Conv3d`）、全连接层（`nn.Linear`）、激活函数（`nn.ReLU`, `nn.Sigmoid`, `nn.Tanh`）等。

3、容器类：

- `nn.Sequential`：允许将多个层按顺序组合起来，形成简单的线性堆叠网络。
- `nn.ModuleList` 和 `nn.ModuleDict`：可以动态地存储和访问子模块，支持可变长度或命名的模块集合。

4、损失函数（Loss Functions）：

- `torch.nn` 包含了一系列用于衡量模型预测与真实标签之间差异的损失函数，例如均方误差损失（`nn.MSELoss`）、交叉熵损失（`nn.CrossEntropyLoss`）等。

5、实用函数接口（Functional Interface）：

- `nn.functional`（通常简称为 `F`），包含了许多可以直接作用于张量上的函数，它们实现了与层对象相同的功能，但不具有参数保存和更新的能力。例如，可以使用 `F.relu()` 直接进行 ReLU 操作，或者 `F.conv2d()` 进行卷积操作。

6、初始化方法：

- `torch.nn.init` 提供了一些常用的权重初始化策略，比如 Xavier 初始化（`nn.init.xavier_uniform_()`）和 Kaiming 初始化（`nn.init.kaiming_uniform_()`），这些对于成功训练神经网络至关重要。

PyTorch torch.nn 模块参考手册

神经网络容器

类/函数	描述
<code>torch.nn.Module</code>	所有神经网络模块的基类。
<code>torch.nn.Sequential(*args)</code>	按顺序组合多个模块。
<code>torch.nn.ModuleList(modules)</code>	将子模块存储在列表中。
<code>torch.nn.ModuleDict(modules)</code>	将子模块存储在字典中。
<code>torch.nn.ParameterList(parameters)</code>	将参数存储在列表中。
<code>torch.nn.ParameterDict(parameters)</code>	将参数存储在字典中。

线性层

类/函数	描述
<code>torch.nn.Linear(in_features, out_features)</code>	全连接层。
<code>torch.nn.Bilinear(in1_features, in2_features, out_features)</code>	双线性层。

卷积层

类/函数	描述
<code>torch.nn.Conv1d(in_channels, out_channels, kernel_size)</code>	一维卷积层。
<code>torch.nn.Conv2d(in_channels, out_channels, kernel_size)</code>	二维卷积层。
<code>torch.nn.Conv3d(in_channels, out_channels, kernel_size)</code>	三维卷积层。
<code>torch.nn.ConvTranspose1d(in_channels, out_channels, kernel_size)</code>	一维转置卷积层。
<code>torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size)</code>	二维转置卷积层。
<code>torch.nn.ConvTranspose3d(in_channels, out_channels, kernel_size)</code>	三维转置卷积层。

池化层

类/函数	描述
<code>torch.nn.MaxPool1d(kernel_size)</code>	一维最大池化层。
<code>torch.nn.MaxPool2d(kernel_size)</code>	二维最大池化层。
<code>torch.nn.MaxPool3d(kernel_size)</code>	三维最大池化层。
<code>torch.nn.AvgPool1d(kernel_size)</code>	一维平均池化层。
<code>torch.nn.AvgPool2d(kernel_size)</code>	二维平均池化层。
<code>torch.nn.AvgPool3d(kernel_size)</code>	三维平均池化层。
<code>torch.nn.AdaptiveMaxPool1d(output_size)</code>	一维自适应最大池化层。
<code>torch.nn.AdaptiveAvgPool1d(output_size)</code>	一维自适应平均池化层。
<code>torch.nn.AdaptiveMaxPool2d(output_size)</code>	二维自适应最大池化层。
<code>torch.nn.AdaptiveAvgPool2d(output_size)</code>	二维自适应平均池化层。
<code>torch.nn.AdaptiveMaxPool3d(output_size)</code>	三维自适应最大池化层。
<code>torch.nn.AdaptiveAvgPool3d(output_size)</code>	三维自适应平均池化层。

激活函数

类/函数	描述
<code>torch.nn.ReLU()</code>	ReLU 激活函数。
<code>torch.nn.Sigmoid()</code>	Sigmoid 激活函数。
<code>torch.nn.Tanh()</code>	Tanh 激活函数。
<code>torch.nn.Softmax(dim)</code>	Softmax 激活函数。
<code>torch.nn.LogSoftmax(dim)</code>	LogSoftmax 激活函数。
<code>torch.nn.LeakyReLU(negative_slope)</code>	LeakyReLU 激活函数。
<code>torch.nn.ELU(alpha)</code>	ELU 激活函数。
<code>torch.nn.SELU()</code>	SELU 激活函数。
<code>torch.nn.GELU()</code>	GELU 激活函数。

损失函数

类/函数	描述
<code>torch.nn.MSELoss()</code>	均方误差损失。
<code>torch.nn.L1Loss()</code>	L1 损失。
<code>torch.nn.CrossEntropyLoss()</code>	交叉熵损失。
<code>torch.nn.NLLLoss()</code>	负对数似然损失。
<code>torch.nn.BCELoss()</code>	二分类交叉熵损失。
<code>torch.nn.BCEWithLogitsLoss()</code>	带 Sigmoid 的二分类交叉熵损失。
<code>torch.nn.KLDivLoss()</code>	KL 散度损失。
<code>torch.nn.HingeEmbeddingLoss()</code>	铰链嵌入损失。
<code>torch.nn.MultiMarginLoss()</code>	多分类间隔损失。
<code>torch.nn.SmoothL1Loss()</code>	平滑 L1 损失。

归一化层

类/函数	描述
<code>torch.nn.BatchNorm1d(num_features)</code>	一维批归一化层。
<code>torch.nn.BatchNorm2d(num_features)</code>	二维批归一化层。
<code>torch.nn.BatchNorm3d(num_features)</code>	三维批归一化层。
<code>torch.nn.LayerNorm(normalized_shape)</code>	层归一化。
<code>torch.nn.InstanceNorm1d(num_features)</code>	一维实例归一化层。
<code>torch.nn.InstanceNorm2d(num_features)</code>	二维实例归一化层。
<code>torch.nn.InstanceNorm3d(num_features)</code>	三维实例归一化层。
<code>torch.nn.GroupNorm(num_groups, num_channels)</code>	组归一化。

循环神经网络层

类/函数	描述
<code>torch.nn.RNN(input_size, hidden_size)</code>	简单 RNN 层。
<code>torch.nn.LSTM(input_size, hidden_size)</code>	LSTM 层。
<code>torch.nn.GRU(input_size, hidden_size)</code>	GRU 层。
<code>torch.nn.RNNCell(input_size, hidden_size)</code>	简单 RNN 单元。
<code>torch.nn.LSTMCell(input_size, hidden_size)</code>	LSTM 单元。
<code>torch.nn.GRUCell(input_size, hidden_size)</code>	GRU 单元。

嵌入层

类/函数	描述
<code>torch.nn.Embedding(num_embeddings, embedding_dim)</code>	嵌入层。

Dropout 层

类/函数	描述
<code>torch.nn.Dropout(p)</code>	Dropout 层。
<code>torch.nn.Dropout2d(p)</code>	2D Dropout 层。
<code>torch.nn.Dropout3d(p)</code>	3D Dropout 层。

实用函数

函数	描述
<code>torch.nn.functional.relu(input)</code>	应用 ReLU 激活函数。
<code>torch.nn.functional.sigmoid(input)</code>	应用 Sigmoid 激活函数。
<code>torch.nn.functional.softmax(input, dim)</code>	应用 Softmax 激活函数。
<code>torch.nn.functional.cross_entropy(input, target)</code>	计算交叉熵损失。
<code>torch.nn.functional.mse_loss(input, target)</code>	计算均方误差损失。

#

```
1 import torch
2 import torch.nn as nn
3
4 # 定义一个简单的神经网络
5 class SimpleNet(nn.Module):
```

```
6         def __init__(self):
7             super(SimpleNet, self).__init__()
8             self.fc1 = nn.Linear(10, 20)
9             self.relu = nn.ReLU()
10            self.fc2 = nn.Linear(20, 1)
11
12            def forward(self, x):
13                x = self.fc1(x)
14                x = self.relu(x)
15                x = self.fc2(x)
16                return x
17
18            # 创建模型和输入
19            model = SimpleNet()
20            input = torch.randn(5, 10)
21            output = model(input)
22            print(output)
```

- 1 如果需要更详细的信息，可以参考 [PyTorch 官方文档] (<https://pytorch.org/docs/stable/nn.html>)。